Cybersecurity

RESEARCH

Open Access

HB-Geo: a street-level IP geolocation method based on hop-constrained subgraphs and binary gates

Zhiyang Zhao[1], Shichang Ding[2], Ruixiang Li[2] and Xiangyang Luo[2,3*]

Abstract

IP geolocation technique determines the geographic location of a target network device based on its IP address, which gives it significant application value in the field of cybersecurity for services such as cyber attack traceback and cybercrime tracking. Client-independent IP geolocation technology can obtain a user's geographic location without requiring active user cooperation. The current state-of-the-art client-independent IP geolocation methods are based on graph learning. In particular, subgraph-based IP geolocation methods have demonstrated significant advantages. However, these methods face two major challenges: (1) if there is no common last-hop router between the target and landmarks, an IP subgraph cannot be constructed, leading to the failure of geolocating some target IPs; (2) the constructed IP subgraphs often contain noisy landmarks, resulting in degraded geolocation accuracy. To address these issues, we propose an IP geolocation method based on hop-constrained subgraphs and binary gates, named HB-Geo. Specifically, we design a novel IP subgraph construction method. The hop-constrained IP subgraphs do not rely on the common last-hop routers, allowing all reachable target IPs to be included in the subgraphs, thereby ensuring successful geolocation for every reachable target IP. Furthermore, we propose a binary gate-based IP geolocation framework. This framework can filter out noisy landmarks through binary gates, enabling target IPs to learn more valuable geographical supervision signals. Experiments on five real-world IPv4/IPv6 networks demonstrate that HB-Geo successfully geolocates all target IPs, while reducing the mean error by 0.36–40.17% and the median error by 1.15–43.88%, showing significant improvements over state-of-the-art baselines.

Keywords IP geolocation, Graph attention networks, Binary gates, IP subgraphs

Introduction

IP geolocation refers to the process of determining the geographic location of a network device using its known IP address (Liu et al. 2026; Li et al. 2024; Lin et al. 2024; Cheng et al. 2025). It is a widely applied technique in the field of cybersecurity (Wang et al. 2019). For example, in network defense, defenders can trace the geographic locations of attackers based on their IP addresses. In cybercrime investigations, investigators can use IP geolocation to identify the sources of cybercriminal information for evidence collection (Ding et al. 2023). In real-world scenarios, many network devices are not equipped with GPS or do not actively report their geographic locations (Zu et al. 2024; Liu et al. 2024). Therefore, client-independent IP geolocation is crucial for security departments. This technique refers to obtaining the geographic locations of users based on their IP addresses without requiring assistance from the users themselves (Tai et al. 2023; Yang et al. 2024; Dan et al. 2021). Currently, the main focus of IP geolocation research is client-independent IP

*Correspondence:
Xiangyang Luo
luoxy_ieu@sina.com
[1] Information Engineering University, Zhengzhou 450001, Henan, China
[2] Key Laboratory of Cyberspace Situation Awareness of Henan Province, Zhengzhou 450001, Henan, China
[3] State Key Laboratory of Mathematical Engineering and Advanced Computing, Zhengzhou 450001, Henan, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

geolocation (Ding et al. 2023; Tai et al. 2023; Yang et al. 2024). For simplicity, this paper refers to client-independent IP geolocation collectively as IP geolocation.

IP geolocation techniques are categorized into database-based, data mining-based, and network measurement-based methods. Network measurement-based IP geolocation has become a key focus of current research (Zu et al. 2022; Wang et al. 2019; Ortega-Velázquez et al. 2025). These methods involve conducting network measurements on landmarks and target IPs, followed by analyzing the network measurement data to determine the geographic locations of target IPs. Previous works such as SLG (Wang et al. 2011) and Corr-SLG (Ding et al. (2021)) accomplish geolocation by using predefined rules based on the relationship between network latency and geographic distance. These methods typically depend on experts' empirical observations of networks. However, in street-level IP geolocation scenarios, it is difficult for such empirical rules to accurately capture the non-linear relationship between latency and distance in network measurement data.

Subsequent studies adopt machine learning techniques to learn more complex nonlinear patterns from network measurement data. For example, NN-Geo (Jiang et al. 2016) mainly uses round-trip times (RTTs) as input features to a two-layer neural network for IP geolocation. Furthermore, MLP-Geo (Zhang et al. 2020) incorporates traceroutes into the learning process and employs MLP to better capture nonlinear patterns in network measurement data. Nevertheless, real computer networks are graph structures. Machine learning-based IP geolocation methods cannot effectively model graph structures. For simplicity, in this paper, 'machine learning-based IP geolocation methods' refers to those that are not based on graph learning.

In recent years, deep learning methods have attracted significant attention due to their exceptional learning capabilities (Jo and Cho 2026; Zhang et al. 2025; Wu and Wang 2025). In the field of IP geolocation, graph learning methods are the primary deep learning methods being adopted. Graph Neural Networks (GNNs) are an important class of graph learning methods (Xu et al. 2024). In recent years, the state-of-the-art IP geolocation methods have been based on GNNs, particularly the series of target-centric or router-centric methods. GNN-based IP geolocation methods can map network measurement data into graph structures and leverage graph neural networks to learn the relationships between target IPs and landmarks, thereby achieving higher geolocation accuracy. Representative methods include GNN-Geo (Ding et al. 2023), which reformulates the IP geolocation task as an attributed graph node regression problem. Methods like Graph-Geo (Wang et al. 2022), Trust-Geo (Tai et al. 2023), Ex-Geo (Yang et al. 2024), Uncertainty-Geo (Zhao et al. 2025a), HMC-Geo (Zhao et al. 2025b), Neighbor-Geo (Wang et al. 2025), EB-Geo (Liu et al. 2025), and TRF-Geo (Liu et al. 2026) establish router-centric or target-centric subgraphs to learn landmark representations, enabling more precise IP geolocation. The foundation of these methods is the observation that the Internet's regional management strategy usually results in IP hosts with a common last-hop router being situated in close physical proximity (Wang et al. 2022; Tai et al. 2023; Yang et al. 2024; Zhao et al. 2025a, b; Wang et al. 2025; Liu et al. 2025, 2026). Furthermore, GT-Geo (Zhao et al. 2026) is the first Graph Transformer-based IP geolocation method, and it is also a full graph-based IP geolocation method.

Although router-centric and target-centric IP geolocation methods demonstrate significant advantages over other methods, they face two major challenges in practical applications:

1) *Some reachable target IPs may fail to be geolocated.* If a method cannot provide a geographical location for a measurable target IP, then the target IP is considered ungeolocalizable. In this paper, 'all target IPs' refers to reachable target IPs. Since these IP geolocation methods create subgraphs by identifying the common last-hop routers between target IPs and landmarks, the absence of such common last-hop routers leads to subgraph construction failure. Consequently, the target IPs cannot be geolocated.
2) *Noisy landmarks in constructed subgraphs.* The generated subgraphs often contain noisy landmarks whose irrelevant representations are captured by the target IPs. This interferes with the model's ability to determine the geographic locations of the target IPs, significantly reducing geolocation accuracy.

Additionally, many existing full graph-based IP geolocation methods are transductive learning methods (Zhang et al. 2020; Ding et al. 2023). They are trained on the full network graph. Whenever a new target IP needs to be geolocated, this IP and its network measurement data must be incorporated into the full graph, and the model must be retrained from scratch. However, this retraining process takes a lot of time. It is an important problem if IP geolocation services users want to quickly get geolocation results.

To address these challenges, we design a novel street-level IP geolocation method named HB-Geo. HB-Geo constructs hop-constrained IP subgraphs capable of geolocating all target IPs. At the same time, this subgraph construction method ensures that HB-Geo supports inductive learning. We propose a binary gate-based

method to remove noisy landmarks, thereby enabling target IPs to learn more valuable landmark representations and achieve higher geolocation accuracy.

The main contributions are summarized as follows.

- We propose an IP geolocation method with inductive learning capability named HB-Geo. HB-Geo does not require retraining all subgraphs whenever a new target IP needs to be geolocated. This capability significantly reduces the time required to geolocate target IPs compared to IP geolocation methods that lack inductive learning ability.
- We propose a novel hop-constrained IP subgraph construction method that guarantees the inclusion of every reachable target IP in subgraph formation. Unlike prior works that rely on common last-hop routers-often absent in real networks-our method enables all reachable target IPs to geolocate relevant landmarks based on hop distance, thereby fundamentally overcoming the geolocation failure problem inherent in existing subgraph-based methods. This construction method, for the first time, enables subgraph-based IP geolocation methods to successfully geolocate all target IPs, thereby significantly enhancing their practical application value.
- We propose a binary gate-based IP geolocation framework. This framework establishes a noisy landmark removal mechanism. To the best of our knowledge, we are the first to provide a formal definition of noisy landmarks. Based on this definition, the mechanism can filter out noisy landmarks through binary gates, enabling target IPs to learn more valuable geographical supervision signals. Compared to existing methods, this framework possesses a stronger noise removal capability, thereby significantly improving the geolocation accuracy.
- Experiments on five real-world IPv4/IPv6 networks show that HB-Geo can successfully geolocate all target IPs and significantly improve geolocation accuracy. Specifically, HB-Geo reduces the mean error by 0.36–40.17% and the median error by 1.15–43.88% compared to the SOTA baselines.

Related work

In this section, we mainly introduce IP geolocation methods. Existing IP geolocation methods can be primarily categorized into three types: database-based methods, data mining-based methods, and network measurement-based methods. In addition, we also discuss some important works related to other directions in geolocation. Table 1 presents a comparison among different types of IP geolocation methods.

Database-based methods

Database-based methods typically obtain the geographic location of a target IP by querying commercial databases. Since commercial databases themselves serve as the core engine driving commercial geolocation systems, querying a commercial geolocation system is essentially equivalent to querying its underlying commercial database. Well-known commercial geolocation systems (commercial databases) include IP2location (Liu et al. 2014), IPIP (Zu et al. 2024), and MaxMind (Zu et al. 2024). IP2Location is distinguished by its global IP coverage and offers an exceptionally rich set of queryable fields, including country, region, city, latitude/longitude, postal code, time zone, connection speed, ISP, carrier, and usage type. IPIP focuses on the research, curation, and distribution of IP geolocation and IP profiling data. Its geolocation database is primarily built upon BGP/ASN data and a worldwide network of measurement probes, enabling city-level IP geolocation annotation. MaxMind integrates multiple data sources-including carrier signals, traceroute measurements, and end-user feedback-and achieves higher geolocation accuracy in Europe and North America than in other regions. These commercial geolocation systems (commercial databases) generally cover IP addresses

Table 1 Comparison of IP Geolocation Methods

Method category	Geolocation approach	Advantages	Disadvantages
Database-based Methods	Query commercial geolocation systems or databases to determine target IP locations	Fast query speed	Low geolocation accuracy; High timeliness requirements
Data Mining-based Methods	Mine and analyze massive Internet data to infer target IP locations	Relatively high geolocation accuracy	Large volumes of high-value data are held by major commercial companies, making them difficult for ordinary researchers to access
Network Measurement-based Methods	Analyze network measurement data to determine target IP locations	High geolocation accuracy; Independent geolocation without user cooperation	Significantly affected by network stability

across multiple countries. The primary advantage of querying these systems is the ability to rapidly obtain the geographic location of a target IP. However, these commercial databases can only provide street-level geolocation for a small fraction of targets; for the vast majority of target IPs, they can only offer city-level, state/province-level, or even country-level locations. Consequently, for most target IPs, the geolocation accuracy of database-based methods is generally low. Moreover, commercial databases are subject to strong timeliness constraints-they require continuous maintenance and frequent updates; otherwise, their accuracy will significantly degrade over time.

Data mining-based IP geolocation methods

Data mining-based IP geolocation methods mine and analyze vast amounts of Internet data to determine the geographic locations of target IPs. Checkin-GeoLiu et al. (2014) utilizes geolocation data shared through location-sharing services and login logs from individual computers to get geographic locations. GeoCluster applies IP interpolation techniques to estimate unknown geographic locations within IP address ranges (Padamanabban and Subramanian 2001). Furthermore, ReverseDNS determines IP geolocation by analyzing reverse DNS hostnames (Dan et al. 2021).

Although data mining-based methods can achieve street-level geolocation for many target IPs, substantial amounts of high-value data are often held by large commercial companies. It is difficult for ordinary researchers to collaborate with these major corporations, thereby hindering their access to large volumes of high-value data. Consequently, for ordinary researchers, employing data mining-based IP geolocation methods is relatively challenging.

Network measurement-based IP geolocation methods

Network measurement-based IP geolocation methods determine the geographic location of target IPs by analyzing network measurement data (Eriksson et al. 2010; Katz-Bassett et al. 2006; Zilberman et al. 2024). These methods do not depend on data from large commercial companies and have become a research hotspot in recent years. Network measurement-based IP geolocation methods are divided into city-level methods and street-level methods. While city-level IP geolocation methods have also been studied in recent years (Ortega-Velázquez et al. 2025; Liu et al. 2020), the current research focus has been on street-level IP geolocation methods. This paper primarily investigates network measurement-based street-level IP geolocation methods.

Street-level methods currently achieve the highest geolocation accuracy among network measurement-based IP geolocation methods. These methods are primarily categorized into three groups: rule-based methods, machine learning-based methods, and graph learning-based methods. Table 2 presents key information of various network measurement-based street-level IP geolocation methods.

Table 2 Comparison of network measurement-based street-level IP geolocation methods

Method	Technique category	Year	Graph type	All targets located[a]
SLG (Wang et al. 2011)	Rule-based	2011	–	–
Corr-SLG (Ding et al. 2021)	Rule-based	2021	–	–
NN-Geo (Jiang et al. 2016)	Learning-based	2016	–	✓
MLP-Geo (Zhang et al. 2020)	Learning-based	2020	–	✓
Graph-Geo (Wang et al. 2022)	GNN-based	2022	Router-centric subgraphs	–
GNN-Geo (Ding et al. 2023)	GNN-based	2023	Full graph	✓
Trust-Geo (Tai et al. 2023)	GNN-based	2023	Router-centric subgraphs	–
Ex-Geo (Yang et al. 2024)	GNN-based	2024	Target-centric subgraphs	–
Uncertainty-Geo (Zhao et al. 2025a)	GNN-based	2025	Router-centric subgraphs	–
HMC-Geo (Zhao et al. 2025b)	GNN-based	2025	Router-centric subgraphs	–
Neighbor-Geo (Wang et al. 2025)	GNN-based	2025	Router-centric subgraphs	–
EB-Geo (Liu et al. 2025)	GNN-based	2025	Router-centric subgraphs	–
TRF-Geo (Liu et al. 2026)	GNN-based	2026	Router-centric subgraphs	–
GT-Geo (Zhao et al. 2026)	Graph Transformer-based	2026	Full graph	✓
HB-Geo(ours)	GNN-based	–	Hop-constrained subgraphs	✓

[a] Whether the method can give an estimated location for all given target IPs or not

Representative rule-based IP geolocation methods are SLG (Wang et al. 2011) and Corr-SLG (Ding et al. 2021). Both methods predefine rules relating delay and distance, and identify the most relevant landmarks as the geographic locations of target IPs according to these rules. However, rule-based IP geolocation methods heavily depend on experts' empirical knowledge of network patterns. The relationship between delay and distance is often complex and may not conform well to the rules defined by these two methods. Consequently, rule-based IP geolocation methods often achieve low geolocation accuracy in many cities.

Since machine learning can automatically learn valuable knowledge from network measurement data, several machine learning-based IP geolocation methods have consequently emerged. NN-Geo (Jiang et al. 2016) employs RTT as a feature and builds a two-layer neural network to perform IP geolocation based on machine learning. MLP-Geo (Zhang et al. 2020) incorporates routing paths into the learning process and utilizes MLP to perform IP geolocation. Although machine learning-based IP geolocation methods outperform rule-based methods in certain networks, real-world networks exhibit a graph structure, and machine learning-based methods are not well-suited for modeling graph-structured data.

In recent years, graph learning methods have attracted widespread attention due to their strong learning capability for graph structures (Zheng et al. 2025; Yang and Liu 2026; Andresini et al. 2026). GNN-based IP geolocation methods are currently the most actively researched approaches. GNN can effectively learn the relationships between target IPs and nearby landmarks within graph structures. GNN-Geo (Ding et al. 2023) transforms the network topology within a city into an attributed graph and frames the IP geolocation task as a node regression problem on the graph. GNN-Geo is a method trained on a full graph. Graph-Geo (Wang et al. 2022) constructs router-centric network subgraphs and proposes a GCN with uncertainty awareness to determine the target IP address. It is the first method trained on subgraphs. Trust-Geo (Tai et al. 2023) extends Graph-Geo with additional enhancements. Trust-Geo incorporates uncertainty sources into the learning procedure and approaches GNN-based IP geolocation as a multi-view decision fusion problem. Ex-Geo (Yang et al. 2024) improves upon the methods of Graph-Geo and Trust-Geo. Ex-Geo introduces target-centric IP subgraphs and enhances model interpretability by employing a variational graph information bottleneck strategy. Uncertainty-Geo (Zhao et al. 2025a) is a fine-grained IP geolocation framework based on the Dirichlet network. It re-examines the IP geolocation problem from a classification perspective and leverages the properties of Dirichlet concentration parameters to evaluate prediction reliability. HMC-Geo (Zhao et al. 2025b) is a hierarchical multi-label classification framework for IP region prediction across multiple geographical granularities. It leverages residual connection-based feature extraction, attention prediction units, and a composite loss function with hierarchical constraints to optimize predictions. Neighbor-Geo (Wang et al. 2025) employs reparameterization and supervised contrastive learning to model and selectively emphasize particular node-to-node neighbor dependencies. EB-Geo (Liu et al. 2025) simulates dynamic training environments via energy optimization and Monte Carlo sampling, and incorporates uncertainty quantification to assess model adaptability, thereby reducing geolocation error. TRF-Geo (Liu et al. 2026) leverages topology guidance and reliable filtering. It jointly optimizes graph structure and representation learning via a self-supervised edge model combined with role-aware message propagation, enhancing robustness against noise in IP geolocation tasks. GT-Geo (Zhao et al. 2026) is an IP geolocation method based on Graph Transformer, which captures geographical location supervision signals from long-range landmarks through the Transformer component, thereby achieving high-precision geolocation in few-landmark environments. GT-Geo also designs a Node and Edge Dimension Selection (NEEDS) algorithm to reduce the number of hyperparameter tuning efforts, thereby improving training speed. It is a full graph-based IP geolocation method.

Graph learning-based IP geolocation methods represent the current SOTA methods, achieving the highest geolocation accuracy. However, many graph learning-based IP geolocation methods learn from IP subgraphs. There are two limitations of existing subgraph-based IP geolocation methods:

1. Existing subgraph-based IP geolocation methods cannot geolocate some target IPs. This is because these methods construct subgraphs by finding common last-hop routers between target IPs and landmarks. However, in real networks, in many cases there is no common last-hop router between a target IP and a landmark, leading to geolocation failure.

2. The subgraphs constructed by existing subgraph-based IP geolocation methods often contain noisy landmarks. Although some methods attempt to remove noise, the extent of noise removal still needs improvement.

It should be noted that although full graph-based IP geolocation methods (e.g., GNN-Geo) can geolocate all target IPs, they introduce more noise due to learning on the full graph. For target IPs that can be geolocated by subgraph-based methods, the geolocation accuracy of full graph-based methods is often lower than that of subgraph-based methods. This is because subgraphs

are smaller in scale than the full graph, and there is less noise in subgraphs than in the full graph. In addition, full graph-based IP geolocation methods are trained on the full graph, resulting in significantly higher training GPU memory overhead compared to subgraph-based methods. Furthermore, because full graph-based methods cannot perform inductive learning, they require retraining every time a new target IP needs to be geolocated, which incurs substantial time overhead. For the above three reasons, we mainly focus on subgraph-based IP geolocation methods in this paper. However, existing subgraph-based methods have two limitations, which are the problems we aim to solve.

Therefore, we propose hop-constrained IP subgraphs, which can solve the problem that existing subgraph-based IP geolocation methods cannot geolocate some target IPs, thereby ensuring that all target IPs can be geolocated. We design a binary gate-based IP geolocation framework that can remove geographical location supervision signals from noisy landmarks through binary gates, thereby improving geolocation accuracy.

Other important works

Landmark mining methods

GeoCAM (Li et al. 2021a) is the first method to leverage webcams for landmark mining. It identifies websites containing real-time webcams through heuristic rules and extracts landmark information from them. LandmarkMiner (Li et al. 2021b) identifies IPs with specific hosting services via classifiers, obtains their domain names through DNS, and matches them with institutional domain names to mine street-level landmarks. IPvSeeYou (Gont et al. 2021) exploits WAN MAC identifiers leaked in IPv6 addresses to achieve street-level geolocation. It obtains WAN MACs by scanning router IPv6 addresses and maps them to a BSSID location database to determine the physical location. ProbeGeo (Lin et al. 2024) is a landmark mining framework that systematically mines common landmarks from multi-type device webpages via search engine retrieval and probe landmarks from vantage-point webpages via semi-supervised classification, offering a complete pipeline for automated, large-scale landmark acquisition from web content. IPv6Landmarker (Cheng et al. 2025) mines IPv6 street-level landmarks by associating WAN IPv6 addresses with WiFi BSSIDs via multiple association rules, filtering candidate landmarks through a coordinate clustering algorithm, and updating landmarks based on IPv6 prefix rotation patterns to ensure stability.

Uncertainty propagation

In the field of IP geolocation, several methods have explored uncertainty propagation. Graph-Geo (Wang et al. 2022) employs an uncertainty-aware GNN that builds a probabilistic posterior distribution via reparameterization and continuous normalizing flow to capture network measurement noise. It aggregates IP knowledge with uncertainty modeling to produce robust street-level geolocation predictions. Trust-Geo (Tai et al. 2023) models both aleatoric and epistemic uncertainties in IP geolocation using evidential deep learning with a Normal-Inverse Gamma distribution. It quantifies prediction confidence and adaptively fuses multi-view information based on uncertainty estimates to enhance geolocation trustworthiness. EB-Geo (Liu et al. 2025) leverages energy-based modeling to assess the compatibility between IP features and candidate locations, and utilizes Monte Carlo sampling to quantify the uncertainty arising from network noise. Uncertainty-Geo (Zhao et al. 2025a) leverages a Dirichlet posterior network to quantify data, distribution, and model uncertainties in region prediction. It uses concentration parameters to assess prediction confidence and adopts a 'region-first, coordinate-second' paradigm for reliable IP geolocation. TRF-Geo (Liu et al. 2026) mitigates uncertainty through topology-guided link refinement and adaptive message filtering. It dynamically prunes spurious edges, reweights reliable connections, and selectively aggregates messages based on role-aware propagation to suppress noise and enhance robustness.

Operational deployment

To the best of our knowledge, among the IP geolocation literature in recent years, Graph-Geo (Wang et al. 2022) stands out as the only framework that has been deployed in production, providing online API and offline database services for over 2700 clients through the Aiwen Tech platform.

Graph sparsification

DropEdge (Rong et al. 2020) randomly drops a subset of edges at each training iteration to mitigate overfitting and over-smoothing in deep GNNs. Since the dropping is stochastic and only applied during training, it does not yield a fixed sparse graph for inference. PTDNet (Luo et al. 2021) introduces a trainable denoising mechanism that selectively removes task-unrelated edges before each GNN layer. It generates layer-wise subgraph samples rather than a single sparsified graph structure for final prediction. NeuralSparse (Zheng et al. 2020) learns a sampling network to extract a k-neighbor subgraph for each node, which is then fed into a GNN for representation learning. The sampling process is stochastic and produces varying subgraphs across different forward passes. LAGCN (Chen et al. 2020) refines graph topology by training an edge classifier to predict whether edges

should be added or removed. This approach assumes the original graph is relatively clean and may struggle in highly noisy environments. SuperGAT (Kim and Oh 2021) enhances GAT by introducing a self-supervised objective that regularizes attention scores to align with edge existence. However, this regularization can be misled when the graph contains substantial noise, as it encourages label agreement among connected nodes. SGAT (Ye and Ji 2021) incorporates an L_0-norm penalty into the attention mechanism to encourage edge-level sparsity, assigning a single importance score to each edge across all layers. This allows the model to discard task-irrelevant connections and produce a structurally simplified graph that performs robustly across both homophilic and heterophilic settings. Among these methods, SGAT has received widespread attention in recent years due to its superior performance. However, SGAT cannot be directly adapted to subgraph-based IP geolocation methods. This is because, first, SGAT is designed for a single graph structure, whereas subgraph-based IP geolocation methods require training across multiple subgraphs. Second, SGAT is tailored for node classification tasks, while IP geolocation is essentially a regression task; therefore, we need to design a loss function specifically for multi-subgraph node regression. Furthermore, computational overhead is also an important consideration in IP geolocation tasks. Although SGAT significantly reduces computational costs compared to other graph sparsification methods, there remains room for further improvement.

Explainability

Graph-Geo (Wang et al. 2022) provides interpretability through feature importance analysis, where it masks each class of features and measures their relative impact on geolocation accuracy. This allows identification of which network features contribute most to the final prediction. Trust-Geo (Tai et al. 2023) offers explainability by quantifying prediction confidence through epistemic uncertainty derived from evidential deep learning, providing developers with confidence scores to monitor system performance. However, its primary focus is on uncertainty estimation rather than structural explainability of the prediction process. Ex-Geo (Yang et al. 2024) enhances model explainability by applying a variational graph information bottleneck strategy that learns a sparse adjacency matrix, retaining only the most pertinent landmarks for prediction. The retained subgraph provides a concise explanation of how the model arrives at its geolocation decision. Neighbor-Geo (Wang et al. 2025) enhances interpretability by using case analysis to visualize the influence of different landmarks on predictions, distinguishing between primary and secondary contributing landmarks. This visualization reveals how the model prioritizes nearby landmarks over distant ones for geolocation decisions. TRF-Geo (Liu et al. 2026) provides structural explainability through its topology-guided link refinement and adaptive message filtering modules, which can be visualized via case studies showing how unreliable edges are pruned and reliable propagation paths are preserved. The role-aware message propagation further clarifies which nodes participate in information exchange during inference.

Definition and formulation

Landmarks

Network landmarks refer to nodes with known geographic locations and stable IP identifiers (Cheng et al. 2025). Landmarks can be network devices such as user terminals, webcams, Wi-Fi access points, or servers (Li et al. 2021a; Cheng et al. 2025; Liu et al. 2014). They can provide geographical supervision signals for target IPs in IP geolocation.

Noisy edges and noisy landmarks

Following Ye and Ji (2021), we provide the formal definitions of noisy edges and noisy landmarks as follows:

Given a connected IP subgraph $G_s = (\mathbf{V}_s, \mathbf{E}_s)$ that contains one target IP T_s and a landmark set $\mathbf{LD}_{\mathrm{SET}} = \{\mathrm{LD}_1, \mathrm{LD}_2, \ldots, \mathrm{LD}_\eta\}$ with $\eta \geq 1$, where all landmarks in $\mathbf{LD}_{\mathrm{SET}}$ are connected to the target IP T_s via unweighted edges, and $\mathbf{V}_s$ is the node set of subgraph G_s. Suppose there exists an edge $e_i \in \mathbf{E}_s$ and a corresponding landmark $\mathrm{LD}_i \in \mathbf{LD}_{\mathrm{SET}}$ connected by e_i. Let f be an IP geolocation algorithm. Let err_1 denote the geolocation error of the target IP T_s when e_i participates in the geolocation process. Let err_2 denote the geolocation error of the target IP T_s when e_i does not participate in the geolocation process (i.e., when e_i is removed). If $err_1 - err_2 \geq 0$, then e_i is called a noisy edge in subgraph G_s, and the landmark LD_i connected to e_i is called a noisy landmark in subgraph G_s.

Probing hosts

A probing host is a server installed with network probing tools, whose function is similar to traceroute (Ding et al. 2023). This software is utilized to conduct network measurements on all landmarks and target IPs, thereby acquiring network measurement data.

Topological rank

For a given target IP node v, let all training landmark nodes be sorted in ascending order of their shortest-path hop distance to v. Landmarks sharing the k-th smallest hop distance are assigned topological rank k. Specifically:

1) *Rank 1 (Nearest Landmarks):* the set of training landmarks with the minimum hop distance to v;
2) *Rank 2 (Second Nearest Landmarks):* the set of training landmarks with the second smallest hop distance to v;
3) *Rank 3 (Third Nearest Landmarks):* the set of training landmarks with the third smallest hop distance to v.

IP geolocation problem formulation

IP geolocation is a technique capable of obtaining the geographic location of a target IP. This technique can estimate the geographic location of a target IP by utilizing information from some known geographic location hosts, which are defined as landmarks. Given N landmarks $\{\mathrm{LD}_i\}_{i=1}^{N}$, N_{tg} target IPs $\{\mathrm{T}_i\}_{i=1}^{N_{tg}}$, the attributes of the N landmarks $\mathbf{X}_{\mathrm{LD}} = \{\mathbf{x}_i\}_{i=1}^{N}$, the network measurement data from the probing hosts to the landmarks $\mathbf{M}_{\mathrm{LD}} = \{m_i\}_{i=1}^{N}$, the latitude and longitude of the landmarks $\mathbf{Y}_{\mathrm{LD}} = \{\mathbf{y}_i\}_{i=1}^{N}$, the attribute of the target IPs $\mathbf{X}_{\mathrm{T}} = \{\mathbf{x}_{\mathrm{T}_i}\}_{i=1}^{N_{tg}}$, and the network measurement data from the probing hosts to the target IPs $\mathbf{M}_{\mathrm{T}} = \{m_{\mathrm{T}_i}\}_{i=1}^{N_{tg}}$, the geolocation model HB-Geo$(\cdot)$ can predict the latitude and longitude $\mathbf{Y}_{\mathrm{T}}$ of the target IPs using the above elements:

$$\mathbf{Y}_{\mathrm{T}} = \text{HB-Geo}(\mathbf{X}_{\mathrm{LD}}, \mathbf{M}_{\mathrm{LD}}, \mathbf{Y}_{\mathrm{LD}}, \mathbf{X}_{\mathrm{T}}, \mathbf{M}_{\mathrm{T}}, \Phi), \tag{1}$$

where Φ represents all learnable parameters. The geolocation server S_{Loc} refers to the server on which the IP geolocation program is installed. This server utilizes the network measurement data $\mathbf{M}_{\mathrm{LD}}$ and $\mathbf{M}_{\mathrm{T}}$, the attribute data $\mathbf{X}_{\mathrm{LD}}$ and $\mathbf{X}_{\mathrm{T}}$, and the geographic location data of landmarks $\mathbf{Y}_{\mathrm{LD}}$ to compute the geographic locations $\mathbf{Y}_{\mathrm{T}}$ of the target IPs. Network measurement data is transmitted over the network from the probing hosts to the geolocation server.

Practical deployment

To run HB-Geo, we need at least one probing host and one geolocation server. The probing host is equipped with Traceroute measurement tools and connected to the Internet. The geolocation server is installed with the HB-Geo code and has a GPU that supports deep learning.

Methodology

HB-Geo primarily consists of two parts: 1) establishing hop-constrained IP subgraphs, and 2) designing binary gate-based graph attention networks to remove noisy landmarks and to estimate geolocation. Our method is shown in Fig. 1.

Establishing hop-constrained IP subgraphs

Previous router-centric and target-centric IP geolocation methods construct subgraphs by finding the common last-hop routers between target IPs and landmarks. However, if the common last-hop routers are not found, subgraph construction fails. Consequently, the excluded target IPs cannot receive geographical supervision signals from landmarks, ultimately leading to target IP geolocation failure.

In HB-Geo, the hop-constrained IP subgraphs we construct can ensure that every target IP can be included in a subgraph. The construction method of hop-constrained IP subgraphs mainly involves the following steps:

1) Convert the network measurement data and attribute data of the target IPs and landmarks into a full network graph. We treat each IP from the network measurement data as a node and establish an edge between any two adjacent IPs. If an anonymous router exists between two adjacent IPs, we skip that router and directly connect the two IPs with an edge. For node features, we use the attributes of each node (9-dimensional for IPv4 data, 13-dimensional for

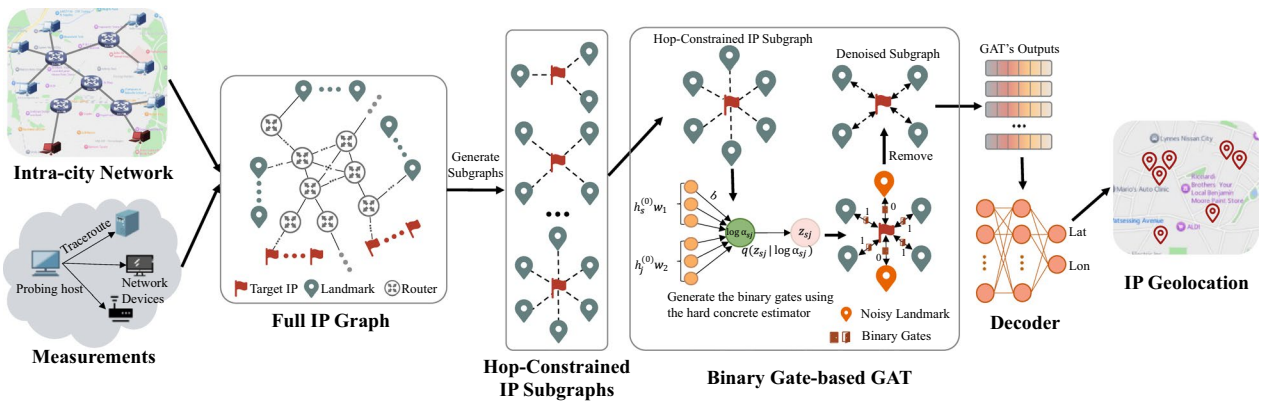


Fig. 1 Overview of HB-Geo

IPv6 data). For the nodes in the node set η, their initial node representations $\mathbf{H}_{\eta}^{(0)}$ are given by:

$$\mathbf{H}_{\eta}^{(0)} = \mathbf{X}_{\eta}\mathbf{Q}_{\eta}, \tag{2}$$

where $\mathbf{Q}_{\eta} \in \mathbb{R}^{J_0 \times J}$ represents the embedding matrix for all nodes η, J_0 denotes the initial embedding dimension size, and J denotes the hidden dimension size.

2) Starting from the target IPs, conduct a full-graph search to calculate the hop counts between the target IPs and all landmarks. Table 3 presents the average of the mean geographical distances between the target IPs and the nearest landmarks. It should be noted that the nearest landmarks in Table 3 refer to those closest to the target IPs in terms of hop count. We can observe that, for the majority of datasets, the closer the target IPs are to the landmarks (the smaller the Rank value), the smaller the average of the mean distances becomes. It is worth noting that in the case of Osaka and Zurich, the average of the mean geographical distances between the second nearest landmarks and the target IPs is slightly smaller than that between the nearest landmarks and the target IPs. Table 4 and Table 5 present the correlation coefficients between Rank values and geographic distance, along with significance tests and ANOVA results. In Table 4, the two coefficients are interpreted as follows: 1. Pearson $r > 0$ and significant: as Rank increases, distance increases linearly; 2. Spearman $\rho > 0$ and significant: as Rank increases, distance increases monotonically. Therefore, our primary focus is on the Spearman correlation coefficient. It can be observed that for the Seoul, Shanghai, Paris, and Zurich datasets, the p-values are all less than 0.05, indicating that the Spearman correlation coefficients are statistically significant. That is, in these datasets, larger Rank values correspond to greater geographic distances. However, we also note that for the Osaka dataset, the p-value is greater than 0.05, suggesting that the Spearman correlation coefficient is not significant in this case-that is, there is no clear evidence that larger Rank values correspond to greater geographic distances in this dataset. This is consistent with our results in Table 2, where for Osaka, the geographic distance corresponding to Rank 1 is actually slightly larger than that corresponding to Rank 2. Nevertheless, the difference between Rank 1 and Rank 2 is minimal. In other words, although hop-constrained IP subgraphs are trained to learn geographic supervision signals from the nearest landmarks in terms of hop count, the quality of the geographic supervision signals from landmarks with different Rank values does not differ substantially in the Osaka dataset due to the absence of a clear monotonic relationship between Rank and distance. Put differently, in Osaka, the performance of hop-constrained IP subgraphs is approximately equivalent to that of other types of subgraphs. Overall, in most cities, the assumption that larger Rank values correspond to greater geographic distances holds. Even in the few cities (e.g., Osaka) where no clear monotonic relationship is observed, the performance of hop-constrained IP subgraphs remains roughly comparable to that of other subgraph types. In summary, the quality of the geographic supervision signals received by hop-constrained IP subgraphs is superior to or at least comparable to that of other types of subgraphs. This also demonstrates the effectiveness of hop-constrained IP subgraphs. Table 5 primarily presents the results of the ANOVA. In Table 5, the F-statistic is defined as the ratio of 'between-group variance' to 'within-group variance', which is used to measure the differences in the mean geographic distances across different Rank groups. A larger F-value indicates that the between-group differences are greater relative to the within-group variability. Levene's Test is employed to assess whether the variances within each group are approximately equal. If the p-value of Levene's Test is greater than 0.05, it suggests that there is no significant difference among group variances, and the assumption of homogeneity of variance holds. It can be observed that for the Seoul, Paris, and Zurich datasets, the p-values of the F-statistic are far below 0.05, while the p-values of Levene's Test are greater than 0.05. This indicates that, for these cities, the differences in geographic distances across different Rank groups are substantially larger than the random fluctuations within each group, and the group variances do not differ significantly. For the Shanghai dataset, although the p-value of Levene's Test is less than 0.05, the conclusion that the differences in geographic distances across different Rank groups are far greater than the within-group random fluctuations remains credible, because the p-value of the F-statistic is extremely small-the difference is too pronounced to be attributed to the confounding effects of unequal variances. For the Osaka dataset, however, since the p-value of the F-statistic is greater than 0.05, there is no significant difference in geographic distances across different Rank groups in Osaka. This indirectly corroborates the results presented in Table 3 and Table 4. Overall, for the majority of cities, the geographic distances across different Rank groups differ significantly. This further corroborates, from another

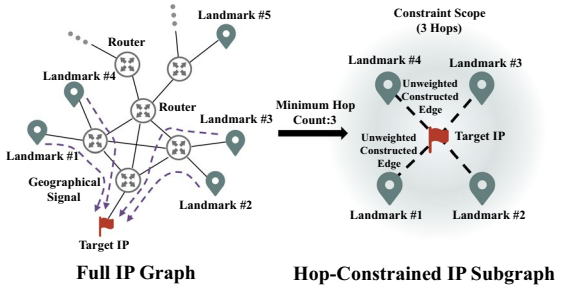


Fig. 2 Generate the hop-constrained IP subgraph

perspective, that the quality of the geographic supervision signals received by hop-constrained IP subgraphs is superior to or at least comparable to that of other types of subgraphs. Therefore, for most cities, the universal pattern still holds: the closer the target IPs are to the landmarks, the smaller the average of the mean geographical distances. Then, we calculate how many hops there are between the target IPs and each landmark. This helps us choose the landmarks we want to use later. It should be noted that we perform traceroute network measurements on the landmarks in our datasets using a probing host located in Hong Kong, China. The obtained network measurement data encompasses different types of networks, different types of routers, network devices with varying densities, and so on. Therefore, our research is inherently oriented toward heterogeneous networks.

3) Select the landmark sets corresponding to the minimum hop counts. Based on the prior observation, the closer the target IPs are to the landmarks, the smaller the average of the mean geographical distances. Therefore, we select all the landmarks from the landmark sets corresponding to the minimum hop counts, aiming to provide more valuable geographical supervision signals for the target IPs.
4) Connect all selected landmarks with the target IPs to form hop-constrained IP subgraphs. After successful subgraph construction, this ensures that every target IP can receive geographical supervision signals from the landmarks, thereby guaranteeing successful geolocation for each target IP.

Table 4 Correlation Analysis Results

City	Spearman's rank correlation		Pearson correlation	
	ρ	p-value	r	p-value
Seoul	0.456543	6.396935×10^{-32}	0.44742	1.402768×10^{-3}
Osaka	−0.009621	9.148529×10^{-1}	0.079359	3.77065×10^{-1}
Shanghai	0.366422	1.49879×10^{-13}	0.315841	2.847277×10^{-1}
Paris (IPv6)	0.542773	1.171305×10^{-4}	0.525864	2.074309×10^{-4}
Zurich (IPv6)	0.268184	1.120451×10^{-5}	0.250557	4.241009×10^{-5}

Table 5 ANOVA and Levene's Test Results

City	F-statistic		Levene's test for homogeneity	
	Value	p-value	Statistic	p-value
Seoul	76.059234	4.040595×10^{-30}	1.280599	2.786407×10^{-1}
Osaka	0.527603	5.913467×10^{-1}	0.99194	3.738034×10^{-1}
Shanghai	24.769518	7.789845×10^{-11}	4.605104	1.056830×10^{-2}
Paris (IPv6)	9.355814	4.360392×10^{-4}	0.801736	4.552945×10^{-1}
Zurich (IPv6)	15.570398	4.128887×10^{-7}	0.60125	5.488921×10^{-1}

We denote each IP subgraph as $G_s = (\mathbf{V}_s, \mathbf{E}_s, \mathbf{H}_s^{(0)})$. Here, $\mathbf{V}_s$ represents the node set of subgraph G_s, $\mathbf{E}_s$ represents the edge set, and $\mathbf{H}_s^{(0)}$ denotes the set of initial node representations. For each IP subgraph $G_s = (\mathbf{V}_s, \mathbf{E}_s, \mathbf{H}_s^{(0)})$, the adjacency matrix $\mathbf{A}_s$ is an unweighted adjacency matrix. Figure 2 illustrates the construction of hop-constrained IP subgraphs. The purple arrows represent the geographical location supervision signals coming from landmarks. The model learns the representations of individual landmarks and infers the geographical location of the target IP based on the geographical locations of these landmarks. The entire flow is shown in Algorithm 1.

Table 3 The average of the mean geographical distances between the target IPs and the nearest landmarks

Landmark type	City				
	Seoul	Osaka	Shanghai	Paris (IPv6)	Zurich (IPv6)
Nearest Landmarks	4.884 km	8.327 km	17.510 km	3.879 km	4.255 km
Second Nearest Landmarks	6.685 km	8.317 km	25.584 km	5.460 km	4.117 km
Third Nearest Landmarks	9.752 km	9.117 km	27.030 km	5.872 km	4.848 km

Algorithm 1 Establishing Hop-Constrained IP subgraphs

Input: $\mathbf{X}_{\mathrm{LD}}, \mathbf{M}_{\mathrm{LD}}, \mathbf{X}_{\mathrm{T}}, \mathbf{M}_{\mathrm{T}}$.
Output: Hop-constrained IP subgraphs $\{G_s\}_{s=1}^{N_{tg}}$.

Use the network measurement data $\mathbf{M}_{\mathrm{LD}}$, $\mathbf{M}_{\mathrm{T}}$ and attribute data $\mathbf{X}_{\mathrm{LD}}$, $\mathbf{X}_{\mathrm{T}}$ to generate the full network graph G_{Full};
for *s in range(1,*$N_{tg}+1$*)***do**
  Compute the hop counts $\mathrm{Hop}_s = \{\mathrm{Hop}_{s1}, \mathrm{Hop}_{s2}, \ldots, \mathrm{Hop}_{sN}\}$ between target IP T_s and all landmarks $\mathrm{LD}_{j=1}^{N}$;
  Find the minimum hop count $\mathrm{Hop}_{smin} = \mathrm{MIN}(\mathrm{Hop}_s)$;
  Extract the index $\mathbf{I}_{s\min} = \{k \in \mathbf{I}_s \mid \mathrm{Hop}_{sk} = \mathrm{Hop}_{smin}\}$ corresponding to the minimum hop count;
  Connect the landmarks corresponding to the index $\mathbf{I}_{s\min}$ with target IP T_s to form a hop-constrained IP subgraph $\{G_s\}$;
end
return $\{G_s\}_{s=1}^{N_{tg}}$;

Binary gate-based IP geolocation

In a hop-constrained IP subgraph, noisy landmarks are likely to provide low-accuracy geographical supervision signals to the target IP. The binary gate aims to identify important edges in the subgraph and remove noisy edges. For an IP subgraph $G_s = (\mathbf{V}_s, \mathbf{E}_s, \mathbf{H}_s^{(0)})$, each binary gate $z_{sj} \in \{0,1\}$ is used to control whether $e_{sj} \in \mathbf{E}_s$ participates in neighborhood aggregation (Ye and Ji 2021). The essence of our noise-robust IP geolocation is to learn a binary gate matrix $\mathbf{Z}_s$ that can set the binary gates of noisy edges to 0, thereby retaining the most critical and useful landmarks. The adjacency matrix $\bar{\mathbf{A}}_s$ with binary gates is defined as:

$$\bar{\mathbf{A}}_s = \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{Z}_s \in \{0,1\}^{N_s \times N_s}, \mathbf{A}_s \in \{0,1\}^{N_s \times N_s}, \quad (3)$$

where N_s represents the number of nodes in subgraph G_s. Regarding subgraph G_s, given the trainable parameters $\mathbf{W} \in \mathbb{R}^{J \times J}$ and binary gates $\mathbf{Z}_s$, we minimize the L_0 -norm to remove unimportant edges as much as possible, forcing the model to learn those truly valuable edges. We present the loss function $\mathcal{L}_s(\mathbf{Z}_s, \mathbf{W})$ for the subgraph G_s. The first term is the geolocation error loss, which adopts the MSE loss. The reason for adopting the MSE loss is that the IP geolocation task is essentially a node regression task, rather than a node classification task. The second term is the L_0-norm loss. It should be noted that this loss function requires further optimization, and we will provide a detailed derivation of the optimization process in the following texts. The loss function $\mathcal{L}_s(\mathbf{Z}_s, \mathbf{W})$ is:

$$\begin{aligned}\mathcal{L}_s(\mathbf{Z}_s, \mathbf{W}) &= \ell(D(f(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{W})), y_s) + \lambda \|\mathbf{Z}_s\|_0 \\ &= \ell(D(f(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{W})), y_s) + \lambda \sum_{(s,j)\in\mathbf{E}_s} |z_{sj}|_0,\end{aligned} \quad (4)$$

where λ is used to balance the weight between the geolocation loss and the L_0-norm loss, D is the decoder that transforms the node representation into the predicted two-dimensional embedding vector of latitude and longitude, y_s is the ground-truth two-dimensional embedding vector of the target IP's latitude and longitude, $\mathbf{H}_s^{(0)} \in \mathbb{R}^{N_s \times J}$ represents the initial node representations of the subgraph G_s, and $f(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{W})$ is the node representation aggregation function.

In $f(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{W})$, we incorporate binary gates into GAT to define the node representation update rule (Ye and Ji 2021). In the constructed hop-constrained IP subgraphs, since target IPs are directly connected to landmarks, a single-layer GAT is sufficient for target IPs to learn geographical supervision signals from the landmarks. Due to the simple structure of hop-constrained IP subgraphs, using a multi-layer GAT may lead to the occurrence of over-smoothing. We set the number of GAT layers to 1. The updated node representation $\mathbf{h}_s^{(1)} \in \mathbb{R}^{1 \times J}$ is computed as follows:

$$\mathbf{h}_s^{(1)} = \sigma\left(\sum_{j \in \mathcal{N}_s} a_{sj} \mathbf{h}_j^{(0)} \mathbf{W}\right), \quad (5)$$

where σ is the sigmoid function, $\mathbf{h}_j^{(0)} \in \mathbb{R}^{1 \times J}$ denotes the initial node representation, and a_{sj} can be represented as:

$$a_{sj} = \mathrm{normalize}(A_{sj} z_{sj}) = \frac{A_{sj} z_{sj}}{\sum_{k \in \mathcal{N}_s} A_{sk} z_{sk}}. \quad (6)$$

Then, we need to minimize the total loss function $\mathcal{L}_s(\mathbf{Z}_s, \mathbf{W})$ to reduce the value of $\|\mathbf{Z}_s\|_0$, thereby removing noisy edges. Since (4) is non-differentiable, to enable the model to learn the corresponding parameters, we

first approximate (4) using the inequality from variational optimization (Bird et al. 2018):

$$\min_{z} \mathcal{F}(z) \leq \mathbb{E}_{z\sim q(z)}[\mathcal{F}(z)]. \tag{7}$$

To make the binary gates differentiable, we can assume that z_{sj} follows a Bernoulli distribution with parameter $\pi_{sj} \in [0,1]$. For the parameter matrix $\boldsymbol{\pi}_s \in [0,1]^{N_s \times N_s}$, only the elements corresponding to existing edges can take non-zero values, e.g., $\boldsymbol{\pi}_{s,j} = \pi_{sj}$. The remaining elements are all zero. According to (7), the upper bound of (4) can be expressed as follows:

$$\begin{aligned}\tilde{\mathcal{L}}_s(\mathbf{W}, \boldsymbol{\pi}_s) = \mathbb{E}_{q(\mathbf{Z}_s|\boldsymbol{\pi}_s)}\ell\big(D(f(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{W})), y_s\big) \\ + \lambda \sum_{(s,j)\in\mathbf{E}_s} \pi_{sj}.\end{aligned} \tag{8}$$

We adopt the hard concrete gradient estimator (Louizos et al. 2018) for further estimation to further optimize (8):

$$\begin{aligned}&\tilde{\mathcal{L}}_s(\mathbf{W}, \log \boldsymbol{\alpha}_s) \\ &= \mathbb{E}_{\boldsymbol{u}_s\sim\mathcal{U}(0,1)}\ell\big(D(f\big(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot g(r(\log \boldsymbol{\alpha}_s, \boldsymbol{u}_s)), \mathbf{W}\big), y_s)\big) \\ &\quad + \lambda \sum_{(s,j)\in E_s} \sigma\big(\log \alpha_{sj} + \beta \log(-\frac{\zeta}{\gamma})\big),\end{aligned} \tag{9}$$

$$\begin{aligned}r(\log \boldsymbol{\alpha}_s, \boldsymbol{u}_s) &= \sigma\Big(\beta^{-1}\big(\log \boldsymbol{u}_s - \log(\mathbf{1} - \boldsymbol{u}_s) + \log \boldsymbol{\alpha}_s\big)\Big) \\ &\quad (\zeta - \gamma) + \boldsymbol{\gamma}_s, g(\cdot) \\ &\quad = \min\big(1, \max\big(0, \cdot\big)\big), \\ &\quad \log \alpha_{sj} = \mathbf{h}_s^{(0)}\mathbf{w}_1 + \mathbf{h}_j^{(0)}\mathbf{w}_2 + b,\end{aligned} \tag{10}$$

where $\mathcal{U}(0,1)$ denotes the uniform distribution over the interval [0, 1], σ is the sigmoid function, $\mathbf{w}_1 \in \mathbb{R}^{J\times 1}$ and $\mathbf{w}_2 \in \mathbb{R}^{J\times 1}$ represent the learnable parameters at the 0-th layer, and b denotes the bias term. For the parameter matrix $\log \boldsymbol{\alpha}_s \in \mathbb{R}^{N_s \times N_s}$, only the elements corresponding to existing edges can take non-zero values, e.g., $\log \boldsymbol{\alpha}_{s,j} = \log \alpha_{sj}$. The remaining elements are all zero. Our design formulates $\log \alpha_{sj}$ as a scalar addition. This formulation reduces the computational cost and thus speeds up the inference process. $\log \boldsymbol{\alpha}_s$ is the logarithmic form that measures the importance scores of edges, and it serves two roles during the training stage: (1) It controls the retention probability of edges. $\log \boldsymbol{\alpha}_s$ determines the probability of edges being retained through the sigmoid function, i.e., $\log \alpha_{sj} + \beta \log(-\frac{\zeta}{\gamma})$. (2) It serves as the basis for reparameterization. $\log \boldsymbol{\alpha}_s$ is used for reparameterized sampling, i.e., $\sigma\big(\beta^{-1}\big(\log \boldsymbol{u}_s - \log(\mathbf{1} - \boldsymbol{u}_s) + \log \boldsymbol{\alpha}_s\big)\big)$. In the hard concrete gradient estimator, β is typically set to $\frac{2}{3}$, γ to -0.1, and ζ to 1.1 (Louizos et al. 2018). The temperature β controls the smoothness of the relaxation: a smaller β makes the sampled values closer to binary, while a larger β makes the distribution more uniform. The stretching parameters γ and ζ ensure that the stretched distribution covers the interval (−0.1,1.1), and the subsequent folding operation maps values in (−0.1,0) to 0 and values in (1,1.1) to 1, enabling exact binary values during inference. The role of $\log \boldsymbol{u}_s - \log(\mathbf{1} - \boldsymbol{u}_s)$ is to introduce random exploration (Louizos et al. 2018), enabling the model to learn which edges are important, while also preventing it from prematurely converging to a local optimum. For the parameter matrix $\boldsymbol{\gamma}_s \in \mathbb{R}^{N_s \times N_s}$, only the elements corresponding to existing edges are equal to γ. The remaining elements are all zero. The function of the hard concrete gradient estimator is reparameterization, which replaces originally discrete variables with continuous variables, thereby making them differentiable.

The decoder D transforms the node representation $\mathbf{h}_s^{(1)} \in \mathbb{R}^{1\times J}$ into the predicted two-dimensional geographic embedding vector $\hat{y}_s$ (latitude and longitude). The decoder D is defined as follows:

$$\hat{y}_s = \text{Sigmoid}(\text{BatchNorm}(\mathbf{h}_s^{(1)}\mathbf{Q}_D + p))), \tag{11}$$

where $\mathbf{Q}_D \in \mathbb{R}^{J\times 2}$ is a linear weight matrix that transforms $\mathbf{h}_s^{(1)}$ into a two-dimensional vector, p is the bias term, BatchNorm denotes batch normalization, and $\hat{y}_s$ represents the predicted latitude and longitude coordinates of the target IP.

For all subgraphs $\{G_s\}_{s=1}^{N_{tg}}$, we take the average of the loss over each subgraph as the total loss. For the geolocation error loss over all subgraphs $\{G_s\}_{s=1}^{N_{tg}}$, we have:

$$\begin{aligned}\tilde{\mathcal{L}}_{MSE} = \frac{1}{N_{tg}} \sum_{s=1}^{N_{tg}} \mathbb{E}_{\boldsymbol{u}_s\sim\mathcal{U}(0,1)} \\ \ell\big(D(f\big(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot g(r(\log \boldsymbol{\alpha}_s, \boldsymbol{u}_s)), \mathbf{W}\big), y_s)\big).\end{aligned} \tag{12}$$

For the L_0 regularization loss over all subgraphs $\{G_s\}_{s=1}^{N_{tg}}$, we have:

$$\tilde{\mathcal{L}}_0 = \frac{1}{N_{tg}} \sum_{s=1}^{N_{tg}} \sum_{(s,j)\in E_s} \sigma\big(\log \alpha_{sj} + \beta \log(-\frac{\zeta}{\gamma})\big), \tag{13}$$

The total loss $\tilde{\mathcal{L}}$ can be expressed as:

$$\tilde{\mathcal{L}} = \tilde{\mathcal{L}}_{MSE} + \lambda\tilde{\mathcal{L}}_0. \tag{14}$$

During the testing phase, we generate a deterministic binary gate matrix $\hat{\mathbf{Z}}_{test}$ based on the trained $\log \boldsymbol{\alpha}_{test}$:

$$\hat{\mathbf{Z}}_{test} = \min\left(1, \max\left(0, \sigma\left(\beta^{-1} \log \boldsymbol{\alpha}_{test}\right)(\zeta - \gamma) + \boldsymbol{\gamma}_{test}\right)\right). \tag{15}$$

After training, some elements of $\hat{\mathbf{Z}}_{test}$ that correspond to existing edges become zero. This indicates that these noisy edges can be removed, thereby preserving landmarks that provide high-accuracy geographical supervision signals. The reason why $\log \boldsymbol{u}_{test} - \log(\mathbf{1} - \boldsymbol{u}_{test})$ is not used when computing $\hat{\mathbf{Z}}_{test}$ is that the primary role of $\log \boldsymbol{u}_{test} - \log(\mathbf{1} - \boldsymbol{u}_{test})$ is to introduce randomness. However, in the testing phase, a deterministic result is needed and, therefore, randomness should not be introduced. It should be noted that whether a landmark is identified as a noisy landmark is determined within a specific subgraph, rather than being judged as a noisy landmark across all subgraphs. This is because HB-Geo learns a binary gate matrix $\hat{\mathbf{Z}}_{test}$ for each subgraph, and these binary gate matrices differ across different hop-constrained IP subgraphs.

It is noteworthy that HB-Geo possesses the capability of inductive learning. Whenever HB-Geo needs to geolocate a new target IP, it does not require retraining. One only needs to construct the $\log \boldsymbol{\alpha}_{test}$ based on the already trained parameters $\mathbf{w}_1$, $\mathbf{w}_2$, and b, and then use the obtained $\log \boldsymbol{\alpha}_{test}$ to construct the binary gate matrix $\hat{\mathbf{Z}}_{test}$ according to (15). Subsequently, we compute $\hat{\tilde{\mathbf{A}}}_{test}$ based on $\hat{\mathbf{Z}}_{test}$ and (3), and then derive the final representation $\hat{\mathbf{h}}^{(1)}_{test}$ of the target IP according to (5). Finally, we obtain the longitude and latitude $\hat{y}_{test}$ of the target IP using (11). Additionally, it should be noted that some shared parameters we set, such as $\mathbf{w}_1$, $\mathbf{w}_2$, b, $\mathbf{W}$, p, and $\mathbf{Q}_D$, serve as an important support for the inductive learning capability of HB-Geo. All subgraphs share these six parameters during training, so that when new subgraphs are added, we can directly use these shared parameters without retraining. It is worth noting that our method also supports multi-head attention. In multi-head attention, for simplicity and to avoid introducing additional parameters, the parameters $\mathbf{w}_1$, $\mathbf{w}_2$, b, p, and $\mathbf{Q}_D$ are shared across the attention heads. Our method is significantly different from some works like GNN-Geo (Ding et al. 2023), which require retraining the full IP graph every time a new target IP is geolocated. Since HB-Geo supports inductive learning, it possesses greater practical value in addressing IP geolocation tasks with high timeliness requirements. The complete IP geolocation process is shown in Algorithm 2.

Algorithm 2 The complete IP geolocation process

Input: Hop-constrained IP subgraphs $\{G_s\}_{s=1}^{N_{tg}}$, Target IP's network measurement data m_{test}, Target IP's attribute data $\mathbf{x}_{test}$.
Output: Target IP's geographic location y_{test}.

```
// Generate the target IP's hop-constrained IP subgraph
```
Generate the hop-constrained IP subgraph G_{test} from the network measurement data m_{test} and attribute data $\mathbf{x}_{test}$ via Algorithm 1;
```
// The training process
```
Generate the initial representation of each node in the training set according to (2);
for *s in range(1,*$N_{tg}+1$*)***do**
  Combine (5), (6), (9), (10), and (11) to compute the geolocation error loss $\tilde{L}_{MSE_s}$ and L_0 regularization loss $\tilde{L}_{0_s}$ for each subgraph G_s;
  Store $\tilde{L}_{MSE_s}$ into the list LIST_{mse};
  Store $\tilde{L}_{0_s}$ into the list LIST_0;
end
Compute the average geolocation error loss $\tilde{L}_{MSE}$ according to (12);
Compute the average L_0 regularization loss $\tilde{L}_0$ according to (13);
Compute the total loss function according to (14);
Continuously optimize to reduce the total loss function $\tilde{L}$, then jump to step 3 to execute the loop until the set number of training epochs is reached;
```
// The testing process
```
Compute the binary gate matrix $\hat{\mathbf{Z}}_{test}$ according to the trained parameters and (15);
Compute the attention coefficients a_{test_j} according to (6);
Compute the final representation $\mathbf{h}_{test}^{(1)}$ of the target IP according to (5);
Compute the geographic location y_{test} of the target IP according to (11);
return y_{test};

Time complexity

In this subsection, we discuss the time complexity of each component of HB-Geo.

Subgraph construction. For each target IP, HB-Geo performs a BFS search on the full graph $G_{Full} = (V_{Full} + E_{Full})$ to compute the hop counts from the target IP to all landmarks. For each target IP, the time complexity is $O(|V_{Full}| + |E_{Full}|)$. For all target IPs, the total time complexity is $O(N_{tg}(|V_{Full}| + |E_{Full}|))$, where V_{Full} is the node set of the full graph and E_{Full} is the edge set of the full graph.

Training complexity. We first analyze the time complexity of one training epoch. For each subgraph G_s, the single-layer GAT computation consists of three parts: node feature transformation: $O(|V_s|J^2)$; attention coefficient computation: $O(|V_s|J^2)$; and neighborhood aggregation: $O(|E_s|J)$. Thus, the forward propagation complexity for a single subgraph is $O(|V_s|J^2 + |E_s|J)$. We denote the average number of nodes across all subgraphs as $|\overline{V_{all}}|$ and the average number of edges as $|\overline{E_{all}}|$. For all subgraphs, the total time complexity is $O(N_{tg}(|\overline{V_{all}}|J^2 + |\overline{E_{all}}|J))$. We next discuss the overhead of binary gates and hard concrete optimization. The computational overhead introduced by

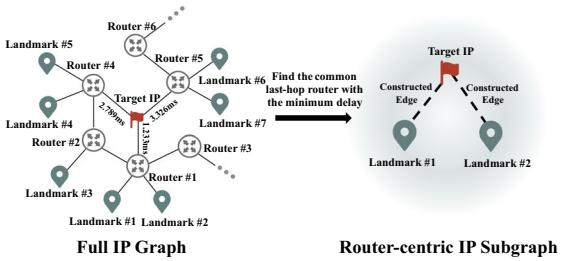


Fig. 3 Generate the router-centric IP subgraph

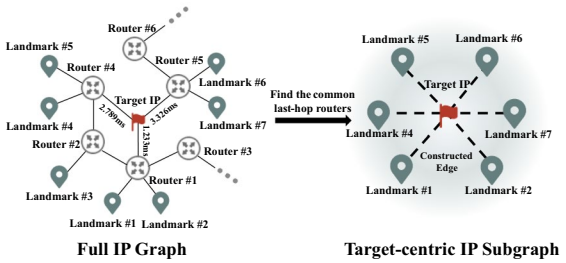


Fig. 4 Generate the target-centric IP subgraph

binary gates and hard concrete optimization mainly comes from two aspects: (1) computing the importance score $\log \alpha_{sj} = \mathbf{h}_s^{(0)}\mathbf{w}_1 + \mathbf{h}_j^{(0)}\mathbf{w}_2 + b$ for each edge in the subgraph; and (2) sampling from the hard concrete distribution during training. For $\log \alpha_{sj}$, each edge requires computing an inner product, resulting in a time complexity of $O(|E_s|J)$. For hard concrete sampling, each edge requires a constant number of basic operations, resulting in a time complexity of $O(|E_s|)$. For all target IPs, the total time complexity introduced by binary gates and hard concrete optimization is approximately $O(N_{tg}|\overline{E_{all}}|J) + O(N_{tg}|\overline{E_{all}}|)$, which simplifies to $O(N_{tg}|\overline{E_{all}}|J)$. Since $O(N_{tg}|\overline{E_{all}}|J)$ is much smaller than $O(N_{tg}(|\overline{V_{all}}|J^2 + |\overline{E_{all}}|J))$, the total time complexity per training epoch remains $O(N_{tg}(|\overline{V_{all}}|J^2 + |\overline{E_{all}}|J))$. Assuming $\hat{T}$ training epochs, the total training time complexity is $O(\hat{T}(N_{tg}(|\overline{V_{all}}|J^2 + |\overline{E_{all}}|J)))$.

Further discussion of our proposed method

In this subsection, we first analyze the advantages of hop-constrained subgraphs over other subgraphs. Then, we further explain why the binary gate-based IP geolocation method can remove noisy landmarks. Finally, we clarify why the hard concrete gradient estimator is chosen as the reparameterization tool.

Advantages of hop-constrained subgraphs over other subgraphs

The advantage of the IP geolocation method based on hop-constrained subgraphs lies primarily in its ability to geolocate target IPs that cannot be geolocated by IP geolocation methods based on router-centric subgraphs or target-centric subgraphs. The IP geolocation methods based on router-centric subgraphs primarily identify the last-hop router with the minimum delay to the target IP, then find landmarks connected to that last-hop router, and establish edges between the target IP and these landmarks to construct a router-centric subgraph. Figure 3 is a schematic diagram of the construction of a router-centric IP subgraph. As shown in the figure, Router #1 is the last-hop router with the minimum delay to the target IP. The target IP finds landmarks connected to Router

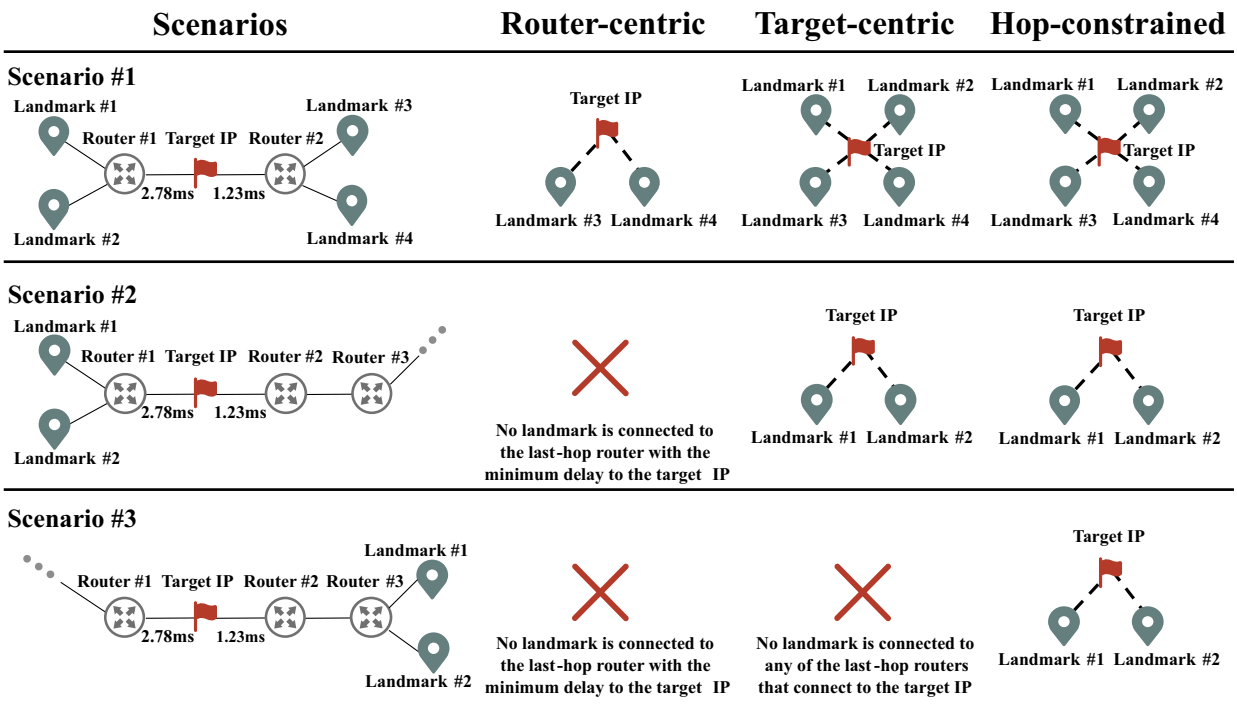


Fig. 5 Construction of different types of subgraphs in various network scenarios

#1 and then establishes edges with them to construct a router-centric subgraph. Representative methods based on router-centric subgraphs include Graph-Geo (Wang et al. 2022), Trust-Geo (Tai et al. 2023), Uncertainty-Geo (Zhao et al. 2025a), HMC-Geo (Zhao et al. 2025b), Neighbor-Geo (Wang et al. 2025), EB-Geo (Liu et al. 2025), and TRF-Geo (Liu et al. 2026).

The IP geolocation method based on target-centric subgraphs primarily finds all last-hop routers connected to the target IP, then identifies landmarks connected to these last-hop routers, and establishes edges between the target IP and these landmarks to construct a target-centric subgraph. Figure 4 is a schematic diagram of the construction of a target-centric IP subgraph. As shown in the figure, the last-hop routers connected to the target IP are Router #1, Router #4, and Router #5. The target IP finds landmarks connected to these routers and then establishes edges with them to construct a target-centric subgraph. The IP geolocation method based on target-centric subgraphs is Ex-Geo (Yang et al. 2024).

However, both router-centric and target-centric IP geolocation methods suffer from a common issue: if a target IP cannot find any landmark that shares a common last-hop router with it, these methods fail to construct a subgraph. For such target IPs, router-centric or target-centric IP geolocation methods cannot perform geolocation. Figure 5 illustrates the construction of different types of subgraphs in various network scenarios.

In Scenario #1: For the router-centric subgraph, the last-hop router with the minimum delay to the target IP is Router #2. There exists landmarks connected to this router. Therefore, the router-centric subgraph can be successfully constructed. For the target-centric subgraph, the last-hop routers of the target IP are Router #1 and Router #2. Landmarks exist connected to both routers. Therefore, the target-centric subgraph can be successfully constructed. For the hop-constrained subgraph, the target IP finds all landmarks with the minimum hop distance. Therefore, the hop-constrained subgraph can be successfully constructed.

In Scenario #2: For the router-centric subgraph, no landmark is connected to the last-hop router with the minimum delay to the target IP. Therefore, the router-centric subgraph fails to be constructed. For the target-centric subgraph, Router #1 remains connected to two landmarks. Therefore, the target-centric subgraph can be successfully constructed. For the hop-constrained subgraph, similar to Scenario #1, the hop-constrained subgraph can be successfully constructed.

In Scenario #3: For the router-centric subgraph, no landmark is connected to the last-hop router with the minimum delay to the target IP. Therefore, the router-centric subgraph still fails to be constructed. For the target-centric subgraph, no landmark is connected to any of the last-hop routers that connect to the target IP. Therefore, the target-centric subgraph fails to be constructed. For the hop-constrained subgraph, similar to Scenarios #1 and #2, the hop-constrained subgraph can be successfully constructed.

As described in Establishing hop-constrained IP subgraphs section, the hop-constrained IP subgraphs we construct can ensure that every target IP can be included in a subgraph. We select all landmarks with the minimum hop count to the target IP to construct a hop-constrained IP subgraph. This ensures that even if a target IP cannot find a landmark sharing the last-hop router with it, it can look for landmarks sharing the last two hops, last three hops, and so on, until landmarks with the minimum hop count to the target IP are found. Through the constructed hop-constrained IP subgraphs, all target IPs are able to learn geographical supervision signals from landmarks. We also show in Table 3 that, on average, landmarks with fewer hops to the target IP are geographically closer to the target IP. In other words, the geographical supervision signals from landmarks with the minimum hop count to the target IP are of higher quality.

Overall, the IP geolocation method based on hop-constrained subgraphs can ensure that all target IPs can be geolocated. It addresses the issue that IP geolocation methods based on other types of subgraphs fail to geolocate some target IPs.

Why the binary gate-based IP geolocation method can remove noisy landmarks

The core idea of this paper is to incorporate binary gates into each edge of the hop-constrained IP subgraph, and to control whether the target IP learns the geographical supervision signal from the corresponding landmark by controlling the binary value of each binary gate. If a landmark is a noisy landmark, its corresponding binary gate should be closed. If a landmark is not a noisy landmark, its corresponding binary gate should be opened, allowing the target IP to learn the geographical supervision signal from that landmark. We incorporate the binary gates into the computation of the loss function, enabling the model to automatically learn the states of the binary gates. In the loss function (4), we design two loss terms, namely $\tilde{L}_{MSEs}$ and $\tilde{L}_{0_s}$:

$$\tilde{L}_{MSEs} = \ell(D(f(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{W})), y_s), \tag{16}$$

$$\tilde{L}_{0_s} = \|\mathbf{Z}_s\|_0. \tag{17}$$

Since the binary gate matrix $\mathbf{Z}_s$ is learnable, it is continuously optimized in the loss term (16) by minimizing the difference between the predicted value

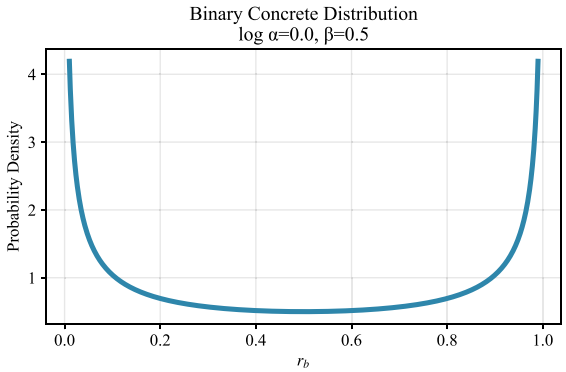


Fig. 6 Illustration of the binary concrete distribution

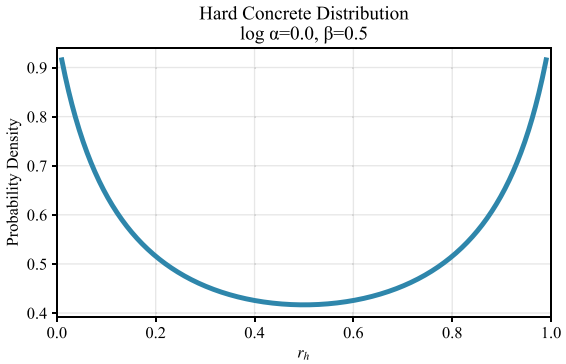


Fig. 7 Illustration of the hard concrete distribution

$D(f(\mathbf{H}_s^{(0)}, \mathbf{A}_s \odot \mathbf{Z}_s, \mathbf{W})$ and the true value y_s. As a result, the binary gates associated with edges connected to noisy landmarks are 'closed', while those connected to non-noisy landmarks are 'opened'. We provide further explanation from another perspective. If a target IP learns geographical supervision signals from noisy landmarks, the predicted geographic location often deviates from its true geographic location by a considerable margin, resulting in a large error gradient for descent. The model minimizes the loss value by making the predicted geographic location as close as possible to the true geographic location, thereby enabling the model to find truly valuable landmarks. We further discuss the design of (17). Incorporating $\|\mathbf{Z}_s\|_0$ into the loss function is intended to keep the value of $\|\mathbf{Z}_s\|_0$ as low as possible. The rationale behind this is to keep as many binary gates closed as possible, thereby filtering out noisy landmarks to the greatest extent and allowing the target IP to learn more valuable geographical supervision signals. Meanwhile, in the loss function (4), λ is used to control the proportion of $\|\mathbf{Z}_s\|_0$, thereby controlling the number of closed binary gates, i.e., how many noisy landmarks are removed. As for why the hard concrete gradient estimator is chosen to reparameterize $\mathbf{Z}_s$, we will discuss this in Why the hard concrete gradient estimator is chosen as the reparameterization tool section.

Why the hard concrete gradient estimator is chosen as the reparameterization tool

In the loss function (4), since the binary gate matrix $\mathbf{Z}_s$ is discrete, it is non-differentiable. However, every variable in the loss function must be differentiable, so we need to reparameterize $\mathbf{Z}_s$ to make it differentiable. Considering that each element of the binary gate matrix $\mathbf{Z}_s$ takes values in $\{0, 1\}$, during reparameterization, the value range of each binary gate needs to fall within the interval $[0, 1]$. Before detailing the advantages of the hard concrete gradient estimator, we first present the binary concrete distribution (Maddison et al. 2016).

The binary concrete distribution is a continuous, differentiable, and reparameterizable distribution with a support of (0, 1). It serves as a continuous relaxation of the discrete Bernoulli distribution, enabling binary gates to be optimized via gradient descent. The sampling formula for the binary concrete distribution is (Maddison et al. 2016):

$$r_b = \sigma\left(\beta^{-1}\left(\log u - \log(1-u) + \log\alpha\right)\right), u \sim \mathcal{U}(0,1), \tag{18}$$

where $\log\alpha$ is the location parameter that controls whether the binary gate tends toward 0 or 1, and β is the temperature parameter. The smaller β is, the closer the sampled value approximates 0 or 1. However, one drawback of the binary concrete distribution is that the sampled value r_b can never take exactly 0 or 1. As shown in Fig. 6, the sampled value r_b can approach 0 or 1 arbitrarily closely, but it can never reach 0 or 1. Therefore, such a distribution cannot accurately characterize whether a binary gate is in the 0 state or the 1 state. In other words, the binary concrete distribution cannot completely filter out noisy edges.

To allow the sampled value to take exactly 0 or 1, we need to stretch the sampling interval, i.e., extend the

Table 6 Basic overview of datasets

Dataset	Landmarks	Nodes	Edges	Size(km^2)
Seoul	1979	3623	4415	605
Osaka	428	749	906	225
Shanghai	1270	2791	4710	6341
Paris (IPv6)	146	200	246	106
Zurich (IPv6)	868	982	1086	92

original (0, 1) interval to include negative values and values greater than 1. To this end, we transform r_b as follows:

$$r_h = r_b(\zeta - \gamma) + \gamma, \tag{19}$$

where γ and ζ are the same as those mentioned in expression (10) above. If we set $\gamma = -0.1$ and $\zeta = 1.1$, the original interval is stretched to $(-0.1, 1.1)$.

We then need to fold the interval $(-0.1, 1.1)$ into $[0, 1]$. When the sampled value r_h lies in the interval $(-0.1, 0)$, we uniformly treat it as 0. When the sampled value r_h lies in the interval (1, 1.1), we uniformly treat it as 1. The expression for folding the interval is as follows:

$$z = \min(1, \max(0, r_h)), \tag{20}$$

where the function of this expression is exactly the function $g(\cdot)$ mentioned in expression (10) above. Through the design of (20), all sampled values in the intervals $(-0.1, 0)$ or (1, 1.1) are set to 0 or 1. This makes the probability of z taking 0 or 1 significantly higher than the probability of taking other values. When performing gradient optimization, the binary gate z can decide whether to remove noisy edges by taking 0 or 1. This distribution is known as the hard concrete distribution (Maddison et al. 2016). Figure 7 shows a schematic diagram of the hard concrete distribution.

Experiments

Experimental settings

Datasets

We utilize five real-world IPv4/IPv6 network datasets: the Seoul dataset (1979 landmarks), Osaka dataset (428 landmarks), Shanghai dataset (1270 landmarks), Paris dataset (146 landmarks), and Zurich dataset (868 landmarks). Among these, the Paris dataset and the Zurich dataset are IPv6 datasets. The landmarks in the Seoul, Osaka, and Shanghai datasets are acquired using GeoCAM (Li et al. 2021a). GeoCAM is an IPv4 landmark mining method that discovers landmarks based on a large number of webcam pages. We conducted on-site validation using GeoCAM in several Chinese cities, and its error was within 30–200 m. The landmarks in the Paris and Zurich IPv6 datasets are obtained using IPvSeeYou (Gont et al. 2021). IPvSeeYou is a landmark mining method specifically designed for IPv6 landmarks. By capitalizing on the unique properties of IPv6 addresses for geolocation, the IPvSeeYou approach creates a large volume of IPv6 street-level landmarks during its operation. We also conducted on-site validation using IPvSeeYou in several Chinese cities, and its maximum error did not exceed 250 m. The detailed contents of the datasets are provided in Table 6. We partition the landmarks in a 7:2:1 ratio, with 70% used for training, 20% for validation, and 10% for testing. In this paper, we use a probing host located in Hong Kong to conduct network measurements on the landmark sets. We use ICMP-Paris probing in our traceroute tool. Using ICMP-Paris probing preserves load balancing.

Baselines

Graph learning-based IP geolocation methods currently represent the SOTA baselines. This paper primarily compares the latest open-source and widely benchmarked SOTA graph learning-based IP geolocation methods.

GNN-Geo (Ding et al. 2023). GNN-Geo is a GNN-based IP geolocation method. It converts a measured intra-city network into an attributed graph, then reframes the IP geolocation problem as a node regression task on this graph. Additionally, GNN-Geo integrates an MLP-based decoder, which leverages rules to improve geolocation accuracy.

Graph-Geo (Wang et al. 2022). Graph-Geo is a router-centric street-level IP geolocation method. It integrates the associations between IP hosts and diverse neighbor types, captures shared domain knowledge and topological features via graph-structured representations, and introduces an uncertainty-aware graph neural network to geolocate the target IP address.

Trust-Geo (Tai et al. 2023). Trust-Geo builds on Graph-Geo with improvements. Trust-Geo integrates uncertainty factors into the learning phase to elevate both the accuracy and reliability of street-level IP geolocation.

Ex-Geo (Yang et al. 2024). Ex-Geo is a target-centric street-level IP geolocation method. Ex-Geo introduces a target-centric IP graph and enhances the model's interpretability through the adoption of a variational graph information bottleneck strategy.

Neighbor-Geo (Wang et al. 2025). Neighbor-Geo leverages reparameterization and supervised contrastive learning to meticulously model. Neighbor-Geo effectively enhances geolocation accuracy by optimizing the selection of neighbor landmarks.

EB-Geo (Liu et al. 2025). EB-Geo simulates dynamic training settings using energy optimization and Monte Carlo sampling, markedly improving the model's robustness and predictive performance. It also integrates uncertainty quantification to gauge adaptability, thus complementing traditional error metrics.

GT-Geo (Zhao et al. 2026). GT-Geo is a graph transformer-based IP geolocation method. It is also a full graph-based IP geolocation method. GT-Geo can learn geographical supervision signals from long-range landmarks through its Transformer component, and it designs a Node and Edge Dimension Selection (NEEDS) algorithm to improve training speed.

Table 7 Performance comparison of different methods across multiple metrics in various cities

Methods	Seoul				Osaka				Shanghai				Paris (v6)				Zurich (v6)			
	MAE	MED	RMSE	GR	MAE	MED	RMSE	GR	MAE	MED	RMSE	GR	MAE	MED	RMSE	GR	MAE	MED	RMSE	GR
Graph-Geo (Wang et al. 2022)	6.166	5.162	7.599	50.5	5.778	5.088	6.873	74.4	14.773	11.081	20.624	49.6	2.751	1.540	3.478	100	3.211	3.065	3.536	97.7
GNN-Geo (Ding et al. 2023)	4.890	3.786	6.321	**100**	5.324	4.829	6.369	**100**	15.385	11.823	19.926	**100**	2.793	2.705	3.038	**100**	3.046	3.158	3.378	**100**
Trust-Geo (Tai et al. 2023)	5.262	3.765	7.076	50.5	6.086	5.913	7.039	74.4	13.517	10.842	18.549	49.6	2.719	0.973	3.841	100	3.211	3.234	3.537	97.7
Ex-Geo (Yang et al. 2024)	5.254	4.148	6.974	50.5	6.208	5.594	7.222	74.4	13.172	8.872	19.389	49.6	3.345	3.338	3.662	100	3.115	3.162	3.43	97.7
Neighbor-Geo (Wang et al. 2025)	5.562	4.065	7.301	50.5	5.633	4.943	6.752	74.4	13.284	10.607	18.218	49.6	2.519	1.308	3.298	100	3.178	3.043	3.484	97.7
EB-Geo (Liu et al. 2025)	5.099	3.614	6.854	50.5	5.518	4.828	6.656	74.4	13.088	10.414	17.954	49.6	2.584	1.311	3.395	100	3.155	3.108	3.482	97.7
GT-Geo (Zhao et al. 2026)	4.925	3.727	6.367	**100**	5.232	4.585	6.364	**100**	13.071	10.224	17.577	**100**	2.091	1.849	2.615	**100**	3.153	3.100	3.446	**100**
HB-Geo	**3.200**	**2.459**	**4.449**	**100**	**5.154**	**4.489**	**6.275**	**100**	**11.536**	**8.347**	**16.365**	**100**	**1.251**	**0.546**	**1.683**	**100**	**3.035**	**3.008**	**3.351**	**100**

Hyperparameter settings

We implement HB-Geo using PyTorch, with two NVIDIA A800 GPUs. We set the hidden layer dimensions to {64, 128}, the number of attention heads to {1, 2, 4}, the learning rates to {0.001, 0.002, 0.005}, λ to 0.01, and the training epochs to 2000. For the baseline methods, we adopt the various hyperparameter combinations provided in their papers and perform hyperparameter tuning on these combinations.

Experimental metrics

The performance of our method is mainly evaluated using three metrics, namely Geolocalizability Rate (GR), Mean Absolute Error (MAE), Median Absolute Error (MED), and Root Mean Square Error (RMSE).

GR. The proportion of the number of target IPs that a geolocation method can give an estimated location in a dataset to the total number of target IPs in that dataset.

MAE. The average distance error between the predicted geographic locations and the actual geographic locations.

MED. The median distance error between the predicted geographic locations and the actual geographic locations.

RMSE. The square root of the average squared distance error between the predicted geographic locations and the actual geographic locations.

Performance comparison

Table 7 shows the comparisons of GR, MAE, MED, and RMSE between HB-Geo and the baselines. In this table, bold indicates the best result and underline indicates the second-best result.

Geolocalizability rate

Table 7 shows that our method HB-Geo maintains 100% GR across all datasets. This is enabled by our proposed hop-constrained IP subgraphs, which incorporate all target IPs into subgraphs. This ensures that all target IPs can learn representations from landmarks, thereby guaranteeing successful geolocation for every target IP.

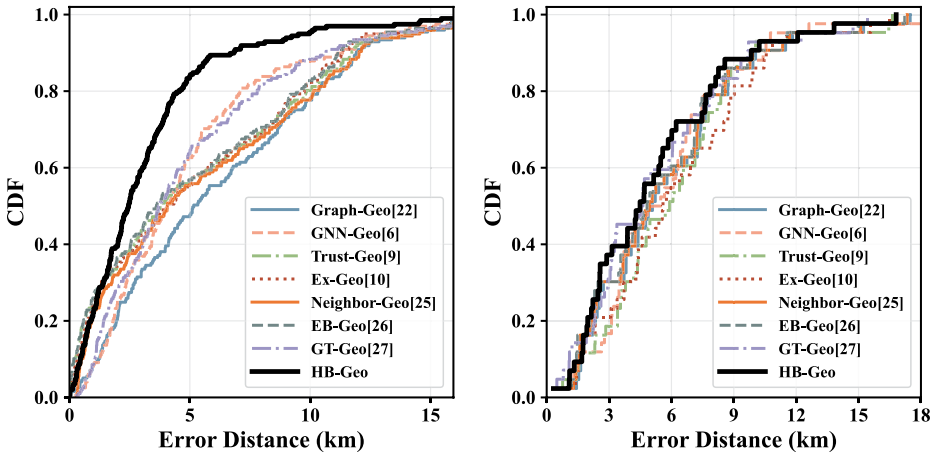
It can be seen that among all baselines, only GNN-Geo and GT-Geo achieves 100% GR. For GNN-Geo and GT-Geo, this is because they adopt a full-graph learning method: all target IPs and landmarks are included in a single graph, so every target IP can learn a representation and thus complete geolocation. In contrast, the other five baselines construct router-centric or target-centric subgraphs. In real-world datasets, many target IPs share no common last-hop routers with any landmark. This prevents those target IPs from being included in subgraph construction, leading to geolocation failures-explaining why these five subgraph-based baselines have low GR values in the Seoul, Osaka, and Shanghai datasets.

Notably, all methods achieve 100% GR in the Paris IPv6 dataset. This is due to the small scale of the dataset: every target IP in it shares a common last-hop router with at least one landmark. Consequently, subgraph-based IP geolocation methods can successfully construct a subgraph for every target IP, thereby achieving a 100% GR. And in the Zurich IPv6 dataset, all subgraph-based IP geolocation methods achieve a GR of 97.7%. This is because the vast majority of target IPs in the Zurich IPv6 dataset each share a common last-hop router with at least one landmark. It can also be observed that the GR of Ex-Geo is equal to that of other router-centric methods. This is because we use a single probe host for network measurement, making the target-centric IP geolocation method functionally equivalent to router-centric methods.

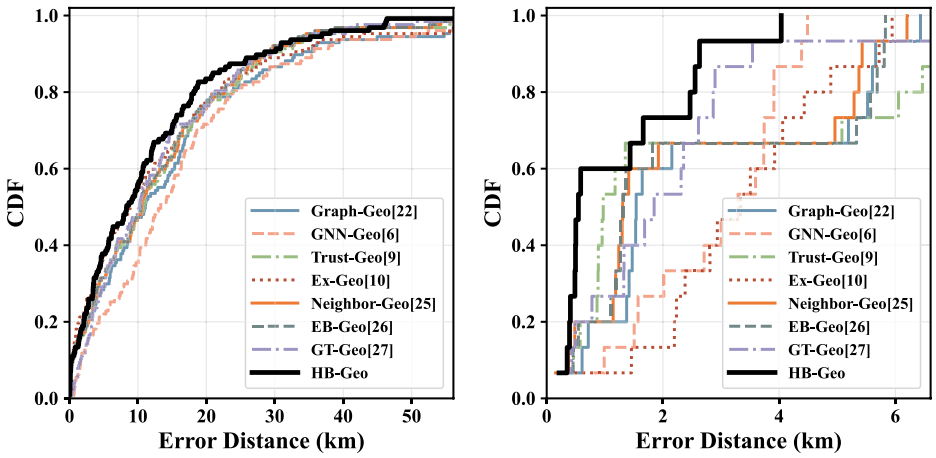
Geolocation accuracy

Since Graph-Geo, Trust-Geo, Ex-Geo, Neighbor-Geo, and EB-Geo are unable to geolocate all target IPs, this paper uniformly predicts the geographic locations of the ungeolocated target IPs (from these methods) as the city center. It can be seen that HB-Geo's geolocation accuracy significantly outperforms the baselines. In the Seoul dataset, HB-Geo achieves a reduction of 34.56% in MAE, 31.96% in MED, and 29.62% in RMSE compared to the SOTA baseline. In the Osaka dataset, HB-Geo achieves reductions of 1.49% in MAE, 2.09% in MED, and 1.40% in RMSE relative to the SOTA baseline. In the Shanghai dataset, HB-Geo lowers MAE by 11.74%, MED by 5.92%, and RMSE by 6.90% compared to the SOTA baseline. In the Paris (IPv6) dataset, HB-Geo demonstrates substantial reductions of 40.17% in MAE, 43.88% in MED, and 35.64% in RMSE against the SOTA baseline. In the Zurich (IPv6) dataset, HB-Geo demonstrates substantial reductions of 0.36% in MAE, 1.15% in MED, and 0.80% in RMSE against the SOTA baseline.

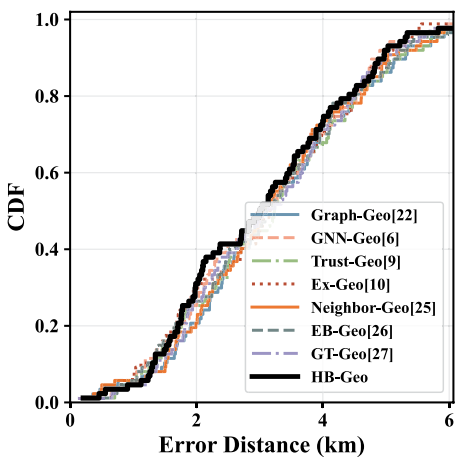
Across different datasets, the magnitude of geolocation error reduction achieved by HB-Geo compared to baselines is different. For example, in the Zurich (IPv6) dataset, the reduction in geolocation error achieved by HB-Geo is relatively small, while in the Seoul dataset, the reduction is relatively large. This may be because in the Zurich (IPv6) dataset, the geographical supervision signals provided by landmarks in the same subgraph have relatively small differences. In other words, there is little difference in the quality of landmarks learned by HB-Geo and other baselines. Furthermore, as can also be seen from Table 3, the average geographical distance from the landmarks with the minimum hop count to the target IPs is slightly larger than that from the landmarks with the



(a) Seoul (b) Osaka



(c) Shanghai (d) Paris(IPv6)



(e) Zurich(IPv6)

Fig. 8 CDF of 5 networks

Table 8 GPU Memory Usage Comparison (in MB)

Method	Seoul	Osaka	Shanghai	Paris(v6)	Zurich(v6)
Graph-Geo (Wang et al. 2022)	534MB	536MB	536MB	542MB	740MB
GNN-Geo (Ding et al. 2023)	2319MB	912MB	2322MB	630MB	986MB
Trust-Geo (Tai et al. 2023)	530MB	530MB	530MB	530MB	552MB
Ex-Geo (Yang et al. 2024)	530MB	530MB	530MB	532MB	562MB
Neighbor-Geo (Wang et al. 2025)	534MB	534MB	534MB	534MB	568MB
EB-Geo (Liu et al. 2025)	530MB	530MB	530MB	532MB	560MB
GT-Geo (Zhao et al. 2026)	3370MB	916MB	2330MB	634MB	993MB
HB-Geo	534MB	532MB	532MB	532MB	536MB

Table 9 Training time comparison of different methods across five datasets

Method	Seoul	Osaka	Shanghai	Paris	Zurich
Graph-Geo	2h4min17s	1min12s	39min32s	2min40s	4min32s
GNN-Geo	6min18s	3min18s	9min2s	2min42s	6min36s
Trust-Geo	21min29s	1min19s	4min14s	39 s	1min7s
Ex-Geo	15min14s	1min37s	3min36s	1min23s	3min47s
Neighbor-Geo	17min38s	2min52s	6min17s	1min42s	5min31s
EB-Geo	19min25s	2min41s	7min56s	1min47s	4min49s
GT-Geo	7min20s	1min58s	7min36s	1min37s	3min12s
HB-Geo	13min22s	2min33s	7min31s	1min39s	8min20s

second minimum hop count. This indirectly corroborates our geolocation results: HB-Geo's hop-constrained IP subgraphs are constructed by connecting target IPs with the nearest landmarks, and HB-Geo cannot learn geographical supervision signals from the second nearest landmarks. The baseline that achieves the smallest geolocation accuracy gap with HB-Geo is GNN-Geo, which is a full graph-based IP geolocation method. GNN-Geo is able to learn geographical supervision signals from the second nearest landmarks, which may be of higher quality than those from the nearest landmarks. Nevertheless, since HB-Geo still achieves higher geolocation accuracy than GNN-Geo, this fully demonstrates the advantage of HB-Geo's denoising capability, while also showing that GNN-Geo introduces more noise when learning higher-quality geographical supervision signals.

In contrast, in the Seoul dataset, the geographical supervision signals provided by landmarks in the same subgraph have relatively large differences. This can make the characteristics of some noisy landmarks more pronounced. HB-Geo is able to effectively remove these noisy landmarks, allowing it to learn landmark quality that is better than other baselines. Furthermore, as can also be seen from Table 3, in the Seoul dataset, the average geographical distance of the landmarks with the minimum hop count is significantly smaller than that of landmarks with other hop counts. This means that the subgraphs constructed by HB-Geo are highly likely to contain high-quality landmarks. As a result, HB-Geo achieves better performance in the Seoul dataset.

Figure 8 presents the CDF curves of HB-Geo compared with six baselines across five networks. In the Seoul dataset, HB-Geo can geolocate 90% of the target IPs with an error of less than 7 km. In the Osaka dataset, HB-Geo keeps the geolocation error within 11 km for more than 90% of the target IPs. For the Shanghai dataset, HB-Geo maintains the error within 18 km for over 80% of the target IPs, and within 25 km for more than 90%. In the Paris (IPv6) dataset, the geolocation error for almost all target IPs is within 4 km. Notably, due to the difficulty in obtaining IPv6 landmark data in Paris and the relatively small number of target IPs, the CDF curve is not smooth. In the Zurich (IPv6) dataset, over 95% of target IPs have a geolocation error within 5 km. In general, HB-Geo significantly outperforms the other baselines, further demonstrating the superior geolocation capability of our method.

GPU memory consumption

Table 8 shows the GPU memory consumption of HB-Geo compared to seven other baselines across different datasets. It can be observed that for IPv4 datasets and the Paris IPv6 dataset, methods trained on subgraphs, such as Graph-Geo, Trust-Geo, Ex-Geo, Neighbor-Geo, EB-Geo, and HB-Geo, exhibit nearly identical GPU memory consumption, and their memory usage remains relatively low. In contrast, GNN-Geo and GT-Geo, which are trained on the full graph, consume significantly higher GPU memory. For the Zurich IPv6 dataset, there are certain differences in GPU memory consumption among different subgraph-based IP geolocation methods. Specifically, Trust-Geo, Ex-Geo, Neighbor-Geo, and EB-Geo have similar memory consumption, while Graph-Geo consumes slightly more GPU memory than the other five

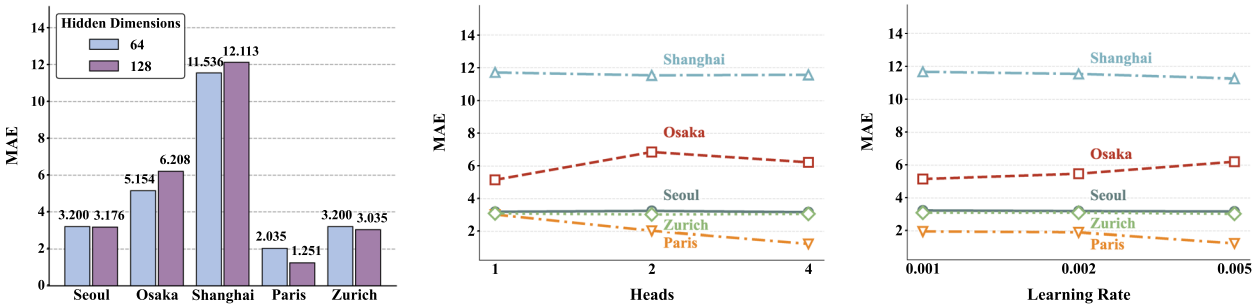


Fig. 9 Parameter sensitivity on five datasets

Table 10 Time Required for Geolocating a New Target IP

Method	Shanghai
GNN-Geo	8min54s
GT-Geo	7min13s
HB-Geo	1.81s

subgraph-based IP geolocation methods. Nevertheless, for both IPv4 and IPv6, the memory consumption of all subgraph-based IP geolocation methods is far lower than that of GNN-Geo and GT-Geo. This is because regardless of the type of subgraph used, the scale of subgraphs is much smaller than that of the full graph. This demonstrates that methods trained on subgraphs are more memory-efficient and impose relatively lower demands on device performance. Furthermore, our proposed method HB-Geo exhibits very small differences in memory consumption across various types of datasets, which fully demonstrates that our method has stronger scalability in terms of memory consumption.

Training time

Table 9 shows the training time comparison of different methods on the five datasets. It can be observed that different methods have their respective advantages in training time across different datasets. With the exception of Graph-Geo, HB-Geo and other methods can complete training within a relatively short time. For Graph-Geo, we can observe that it is particularly sensitive to dataset scale. The impact of dataset scale on Graph-Geo's training time is much greater than that on other methods. Furthermore, we can observe that HB-Geo's training time on the Zurich dataset is longer than that on the Shanghai dataset, even though the scale of the Zurich dataset is smaller than that of the Shanghai dataset. This may be because the average number of landmarks in the hop-constrained subgraphs of the Zurich dataset is far greater than that in the Shanghai dataset. According to Time complexity section, the training time complexity of HB-Geo is positively correlated with the number of landmarks and edges in the subgraphs. Consequently, this leads to HB-Geo's training time on the Zurich dataset being longer than that on the Shanghai dataset. Overall, the training time of our method HB-Geo on all datasets is within 15 min, which is acceptable for practical geolocation tasks.

Experimental validation of inductive learning advantages

We use the Shanghai dataset as a case study to demonstrate the time efficiency advantage of HB-Geo over transductive methods when geolocating a new target IP. Table 10 presents a comparison of the time consumed by HB-Geo, GNN-Geo, and GT-Geo when geolocating a new target IP.

It can be observed that HB-Geo, owing to its inductive learning capability, does not require retraining when geolocating a new target IP and thus consumes only 1.81s. In contrast, GNN-Geo and GT-Geo are full graph-based IP geolocation methods that lack inductive learning capabilities. When a new target IP is added, the full network graph changes, necessitating retraining of the entire graph. As shown, the transductive methods GNN-Geo and GT-Geo consume 8min54s and 7min13s, respectively, far exceeding the time required by HB-Geo. These experimental results fully demonstrate the advantages of HB-Geo's inductive learning capability.

Parameter sensitivity

Figure 9 demonstrates the parameter sensitivity of hidden layer embedding dimensions, number of attention heads, and learning rates across five datasets. It can be observed that in the Seoul dataset, the MAE values corresponding to the three parameters show no significant changes, indicating that these three parameters are not sensitive. In Osaka, the optimal hidden layer embedding

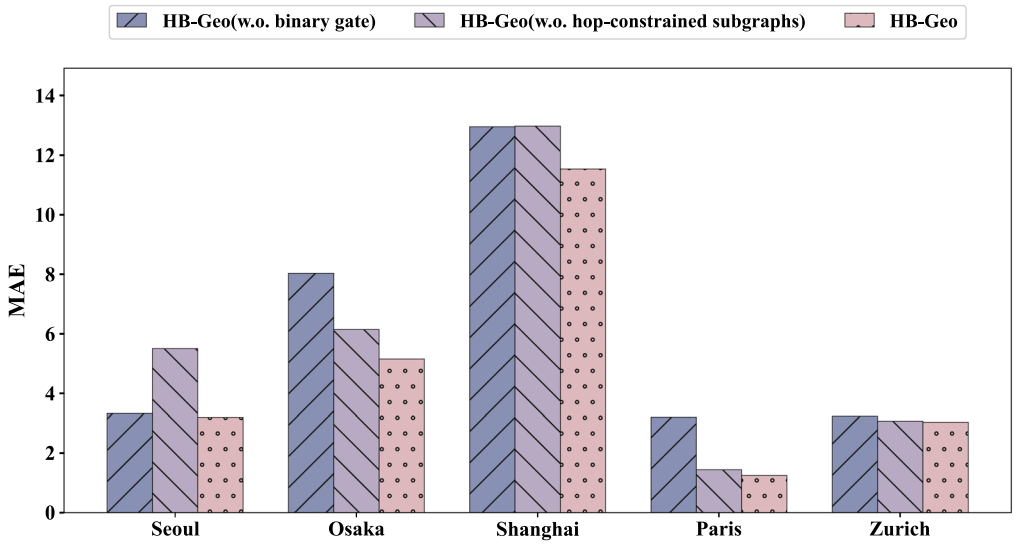


Fig. 10 Ablation studies on five datasets

dimension is found to be 64, and the optimal number of attention heads is found to be 1. Regarding the learning rate, the lower the learning rate, the higher the geolocation accuracy, with the optimal learning rate being 0.001. In Shanghai, the optimal hidden layer embedding dimension is found to be 64. For the number of attention heads, the differences in geolocation error across different numbers of heads are relatively small, and the optimal number is found to be 2. As for the learning rate, the geolocation accuracy increases with the learning rate, and the optimal value is found to be 0.005. In the Paris (IPv6) dataset, the optimal hidden layer embedding dimension is found to be 128. In terms of the number of attention heads, the higher the number of heads, the higher the geolocation accuracy, with the optimal number of attention heads being 4. With respect to the learning rate, higher values are associated with better performance, and the most effective learning rate is determined to be 0.005. In the Zurich (IPv6) dataset, the optimal hidden layer embedding dimension is also 128. Regarding the number of attention heads, varying the head count does not lead to significant differences in accuracy, and the optimal configuration is achieved with 2 heads. As for the learning rate, a larger value yields better performance, and the optimal learning rate is found to be 0.005.

Ablation study

We perform ablation studies to validate the effectiveness of the individual components within HB-Geo. Figure 10 demonstrates how hop-constrained IP subgraphs and binary gates influence the geolocation accuracy of HB-Geo. For HB-Geo without hop-constrained IP subgraphs, we adopt router-centric IP subgraphs as a replacement. It can be observed that both hop-constrained IP subgraphs and binary gates play significant roles in improving geolocation accuracy. Notably, the effect of binary gates is not very pronounced in the Seoul dataset. This may be because in the Seoul dataset, the differences in importance among landmarks within the subgraphs are not significant, meaning there are relatively few or no obvious noisy landmarks. Therefore, our method removes only a small number of noisy landmarks, resulting in minimal structural changes to the subgraphs before and after noise removal. Consequently, the difference in final geolocation accuracy is not substantial. Furthermore, we can observe that in the Paris IPv6 and Zurich IPv6 datasets, the effect of hop-constrained subgraphs is not very pronounced. This phenomenon can be easily explained: in these two IPv6 datasets, other subgraph-based IP geolocation methods achieve geolocalizability rates (GR) of 100% and 97.7%, respectively, which are significantly higher than those in other datasets. Since the core advantage of hop-constrained subgraphs over other subgraphs is their ability to geolocate all target IPs, the effect of hop-constrained subgraphs is less prominent in these datasets.

Discussion

Scalability

In this subsection, we discuss scalability for Internet-scale datasets or large autonomous systems. First, since HB-Geo and other baseline methods all perform street-level IP geolocation based on known city-level geographic

Table 11 GPU memory consumption on the large-scale dataset

Method	Los angeles
Graph-Geo	3122MB
GNN-Geo	58656MB
Ex-Geo	2422MB
GT-Geo	58733MB
HB-Geo	1599MB

Table 12 Training time consumption on the large-scale dataset

Method	Los angeles
Graph-Geo	2d2h22m8s
GNN-Geo	2d14h48m
Ex-Geo	1d6h38m49s
GT-Geo	4d9h43m28s
HB-Geo	7h20m12s

locations of target IPs, we can discuss scalability within a large-scale city dataset or a large autonomous system. Scalability can be analyzed from two perspectives: GPU memory consumption and training speed.

We first discuss the advantages of HB-Geo in terms of GPU memory consumption compared to other baseline methods. Compared with full graph-based IP geolocation methods, since full graph-based methods are trained on the full graph while HB-Geo is trained on hop-constrained IP subgraphs, and the scale of the full graph is much larger than that of the subgraphs, full graph-based methods generally consume significantly more memory. This is also demonstrated in Table 8. Therefore, from a GPU memory consumption perspective, HB-Geo exhibits much stronger scalability to Internet-scale datasets and large autonomous systems than full graph-based IP geolocation methods.

We then compare HB-Geo with other subgraph-based IP geolocation methods. Since the scale differences among various types of subgraphs are very small-almost negligible-the GPU memory consumption among them is also nearly indistinguishable. For example, in Table 8, across different datasets, the GPU memory consumption of all subgraph-based IP geolocation methods remains between 530 MB and 740 MB. This demonstrates that the scale differences among different types of subgraphs are minimal, and the subgraph scale is almost independent of the dataset size. This indicates that, theoretically, our method HB-Geo and other subgraph-based IP geolocation methods can be deployed on datasets of arbitrary scale without causing memory explosion. In contrast, full graph-based IP geolocation methods experience increasing memory consumption as dataset scale grows, making it difficult for them to scale to large datasets.

Another aspect is training time. For both full graph-based IP geolocation methods and subgraph-based IP geolocation methods, training time increases as the dataset scale grows. The significant advantage of HB-Geo over full graph-based IP geolocation methods lies in its inductive learning capability. That is, when a new target IP needs to be geolocated, full graph-based methods require retraining, which consumes substantial time. From this perspective, HB-Geo exhibits stronger scalability on very large-scale datasets compared to full graph-based methods.

However, since HB-Geo is a subgraph-based IP geolocation method, a very large number of subgraphs will be constructed on large-scale datasets, and training on such a large number of subgraphs inevitably incurs increased time overhead. Nevertheless, this limitation is not unique to HB-Geo but is a common issue faced by graph learning-based IP geolocation methods in general. It should also be noted that although a few traditional rule-based IP geolocation methods require no training, their geolocation accuracy is often significantly weaker than that of graph learning-based IP geolocation methods (as demonstrated in several papers, including GNN-Geo (Ding et al. 2023) and GT-Geo (Zhao et al. 2026)).

To verify the scalability of HB-Geo on large-scale datasets, we add experiments on a large-scale dataset. We select the Los Angeles dataset, one of the largest publicly available datasets known to date, for our experiments. The Los Angeles dataset contains 92,804 landmarks. We select the router-centric IP geolocation representative method Graph-Geo, the target-centric IP geolocation representative method Ex-Geo, and the full graph-based IP geolocation methods GNN-Geo and GT-Geo as baselines. Table 11 presents the GPU memory consumption comparison on the large-scale dataset. It can be observed that even on such a large-scale dataset, HB-Geo consumes only 1,599 MB of GPU memory. Combined with Table 8, we can see that HB-Geo's GPU memory consumption grows slowly despite the substantial increase in dataset scale. This indicates that HB-Geo maintains a relatively low level of GPU memory consumption on large-scale datasets, demonstrating good scalability. In contrast, we can see that the full graph-based IP geolocation methods GNN-Geo and GT-Geo consume extremely high GPU memory on the large-scale dataset. Once the dataset scale increases further, GNN-Geo and GT-Geo suffer from memory explosion and fail to complete geolocation. We can also observe that the GPU memory consumption of subgraph-based IP geolocation methods on the large-scale dataset is far lower than that of full graph-based

IP geolocation methods, which fully demonstrates the advantages of subgraph-based IP geolocation methods in terms of scalability.

Table 12 presents the training time consumption comparison of each method on the large-scale dataset. It can be observed that HB-Geo consumes only 7h20m12s in training time, which is entirely acceptable for a large-scale dataset. Meanwhile, we notice that the training time consumption of GNN-Geo and GT-Geo is significantly higher than that of the other methods. This is because GNN-Geo and GT-Geo consume very high GPU memory; therefore, under limited GPU memory resources, parallel hyperparameter tuning and training are infeasible for them. In contrast, other methods, due to their lower memory consumption, can fully support parallel hyperparameter tuning and training, thus consuming less time compared to GNN-Geo and GT-Geo. Furthermore, thanks to HB-Geo's inductive learning capability, when geolocating a new target IP, no retraining is required. Therefore, from the perspective of training time consumption, the experimental results also sufficiently demonstrate HB-Geo's advantages in scalability.

In summary, how to reduce GPU memory consumption while improving training speed on large-scale datasets is a very important problem, and it is also a key direction for our future research, thereby making IP geolocation methods more practical.

Potential issues in practical deployment

In practical deployment, the following potential issues may arise: MPLS tunnels, VPNs, CDNs, and so on. We discuss each of these issues separately.

First, MPLS tunnels. MPLS tunnels prevent traditional traceroute measurements from revealing the complete routing path, because intermediate nodes in MPLS label-switched paths typically do not appear in traceroute output. This causes HB-Geo to lose some intermediate nodes when constructing the full graph, thereby affecting the accuracy of hop count calculation. This is a limitation of our current method, as the network measurement data used in this paper is based on standard traceroute and cannot directly perceive MPLS tunnels. This is a problem that affects the entire IP geolocation field. In future work, we will incorporate considerations of MPLS tunnels.

Second, VPNs. When a target IP is located behind a VPN, traceroute measurements only reach the VPN gateway. Inside the VPN tunnel, intermediate routers are invisible to traceroute because the original packets are encapsulated and encrypted. Consequently, the measured hop count to the target IP is underestimated, and HB-Geo may select landmarks based on this inaccurate hop count, potentially biasing the geolocation result toward the VPN gateway's geographic location rather than the user's true physical location. To date, no method in the current field of IP geolocation has considered VPNs.

Third, CDNs. CDN nodes often use anycast IP technology, where the same IP address may appear in multiple geographical locations simultaneously, which may introduce certain errors in IP geolocation. This is also a common problem faced by all current IP geolocation methods. In the future, we can combine DNS resolution information, active probing, or application-layer features to identify CDN nodes and adopt different processing strategies.

Security analysis of adversarial manipulation

Since network measurement-based IP geolocation methods all rely on landmarks and network measurements, the geolocation accuracy of existing network measurement-based IP geolocation methods would be severely affected by adversarial manipulation of routing paths or landmarks. Specifically, adversaries could replace the geographic locations of landmarks, causing landmarks to provide false geographical supervision signals. Additionally, adversaries could interfere with network routing or configure settings on routers or terminals to transmit false network measurement data to probing hosts. Network measurement-based IP geolocation methods can only utilize network measurement data and geographical supervision signals from landmarks for geolocation, thereby making their geolocation accuracy vulnerable to such interference. This is a common limitation faced by all existing network measurement-based IP geolocation methods. To overcome this problem, in future work, we will explore how to conduct zero-trust IP geolocation. Specifically, we will consider scenarios where landmarks are untrustworthy and network measurement data is untrustworthy, and introduce zero-trust theory into IP geolocation. Furthermore, we may even attempt to perform IP geolocation without any landmarks at all, thereby completely addressing the issue of adversarial manipulation of routing paths or landmarks.

Degradation scenarios

There are two main scenarios where our method may significantly degrade: one is the scenario with extremely sparse landmarks, and the other is the scenario with unreliable landmarks and unreliable network measurement data.

Scenario 1: Extremely few landmarks for a large number of target IPs. When the number of landmarks is very small, landmarks are often extremely sparse in the network relative to the target IPs. In this scenario, the landmarks with the minimum hop count to a target IP

are likely to be geographically far away, leading to inaccurate geolocation. Furthermore, with very few landmarks, the target IP can learn very limited geographical supervision signals. Graph learning-based IP geolocation methods typically require a sufficient number of geographical supervision signals. Insufficient supervision signals inevitably cause significant geolocation errors. In an extreme case where a hop-constrained IP subgraph contains only one landmark, the target IP can only learn the geographical supervision signal from that single landmark, even if that landmark is far away from the target IP. Moreover, in this case, the binary gate mechanism will fail and cannot perform denoising. Because if the only landmark is removed as a noisy landmark, the target IP would have no geographical supervision signal to learn from in that subgraph. Therefore, in this extreme case, regardless of whether the landmark provides high-quality or low-quality geographical supervision signals, the target IP must forcibly rely on the geographic location information of this landmark, which will affect the geolocation accuracy of that target IP.

Scenario 2: Unreliable landmarks and unreliable network measurement data. As discussed in Security analysis of adversarial manipulation section, adversaries can maliciously manipulate landmarks or routing paths, making both landmarks and network measurement data unreliable. Our method relies on network measurement data and assumes that the geographic locations of landmarks are completely authentic. However, if adversaries maliciously modify the geographic locations of landmarks, the geolocation accuracy of our method will significantly degrade.

Both of the above scenarios may significantly degrade the geolocation accuracy of our method. In future work, we will focus on exploring how to address the challenges posed by these two scenarios. For example, in the scenario with extremely sparse landmarks, we can consider expanding the perception range of the hop-constrained IP subgraph-that is, not limiting ourselves to landmarks with the minimum hop count, but expanding the perception range to learn more geographical supervision signals, thereby mitigating the issues caused by sparse landmarks. For the scenario with unreliable landmarks and unreliable network measurement data, as discussed in Security analysis of adversarial manipulation section, we can introduce zero-trust theory into IP geolocation, and may even attempt to perform IP geolocation without any landmarks at all in future work.

Conclusion

We propose a street-level IP geolocation method based on hop-constrained subgraphs and binary gates named HB-Geo. This method constructs hop-constrained IP subgraphs, which ensure that every target IP can be successfully geolocated. Furthermore, it leverages binary gates to determine which noisy landmarks should be removed, thereby allowing target IPs to learn more valuable landmark information. Experiments conducted on five real-world IPv4/IPv6 networks validate that HB-Geo achieves outstanding performance, significantly improving IP geolocation accuracy. For future work, we can incorporate additional edge features into the IP subgraphs to enable target IPs to learn more accurate landmark information. Furthermore, we will take into account traceroute stability and anonymous routers, while simultaneously considering the geolocation of unreachable target IPs. We will also consider addressing the issues caused by MPLS tunnels, VPNs, and CDNs in IP geolocation. In addition, we will explore introducing zero-trust theory into IP geolocation to mitigate the issue of adversarial manipulation of routing paths or landmarks, and investigate how to achieve high-precision geolocation in scenarios with extremely sparse landmarks.

Acknowledgements
Not applicable.

Author Contributions
Zhiyang Zhao is responsible for methodology design and manuscript writing. Shichang Ding and Ruixiang Li are responsible for providing revision suggestions for the manuscript. Xiangyang Luo is responsible for providing revision suggestions for the manuscript and funding support.

Funding
This work was supported by the National Natural Science Foundation of China (No. U23A20305, No. 62302521), the Innovation Scientists and Technicians Troop Construction Projects of Henan Province (No. 254000510007).

Data availability
The code and datasets are available at https://github.com/LuckyBoy0517/HB-Geo

Declarations

Ethics approval and consent to participate:
Not applicable.

Consent for publication:
All authors consented to the publication of this manuscript.

Competing interests:
There are no conflict of interest for this manuscript.

Received: 6 April 2026 Accepted: 31 August 2026
Published online: 09 September 2026

References

Andresini G, Appice A, Belvedere V, Fiameni G, Malerba D (2026) Anakin: explainable android malware detection with graph neural networks. Cybersecurity 9(1):116

Bird T, Kunze J, Barber D (2018) Stochastic variational optimization. arXiv:1809.04855 arXiv preprint

Chen, H., Xu, Y., Huang, F., Deng, Z., Huang, W., Wang, S., He, P., Li, Z.: Label-aware graph convolutional networks. In: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, pp. 1977–1980 (2020)

Cheng R, Ding S, Zhang L, Li R, Du S, Luo X (2025) Ipv6landmarker: enhancing ipv6 street-level geolocation through network landmark mining and targeted updates. IEEE Trans Netw Sci Eng 12(2):1280–1296

Dan O, Parikh V, Davison BD (2021) Ip geolocation through reverse dns. ACM Trans Internet Technol (TOIT) 22(1):1–29

Dan O, Parikh V, Davison BD (2021) Ip geolocation using traceroute location propagation and ip range location interpolation. Companion Proceedings of the Web Conference 2021. pp 332–338

Ding S, Zhao F, Luo X (2021) A street-level ip geolocation method based on delay-distance correlation and multilayered common routers. Secur Commun Netw 2021(1):6658642

Ding S, Luo X, Wang J, Fu X (2023) Gnn-geo: a graph neural network-based fine-grained ip geolocation framework. IEEE Trans Netw Sci Eng 10(6):3543–3560

Eriksson B, Barford P, Sommers J, Nowak R (2010) A learning-based approach for ip geolocation. International Conference on Passive and Active Network Measurement. Springer, pp 171–180

Gont F, Krishnan S, Narten T, Draves R (2021) Temporary address extensions for stateless address autoconfiguration in IPv6. Internet Engineering Task Force RFC, Fremont, CA, USA

Jiang H, Liu Y, Matthews JN (2016) Ip geolocation estimation using neural networks with stable landmarks. In: 2016 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS). IEEE, pp 170–175

Jo S, Cho Y (2026) An enhanced backdoor attack using a backdoor trigger position searching algorithm for avoiding deep learning-based object detection systems. Cybersecurity 9(1):128

Katz-Bassett E, John JP, Krishnamurthy A, Wetherall D, Anderson T, Chawathe Y (2006) Towards ip geolocation using delay and topology measurements. In: Proceedings of the 6th ACM SIGCOMM Conference on Internet Measurement. pp 71–84

Kim, D., Oh, A.: How to find your friendly neighborhood: Graph attention design with self-supervision. In: 9th International Conference on Learning Representations, ICLR 2021 (2021)

Li Q, Wang Z, Tan D, Song J, Wang H, Sun L, Liu J (2021) Geocam: an ip-based geolocation service through fine-grained and stable webcam landmarks. IEEE/ACM Trans Netw 29(4):1798–1812

Li R, Xu R, Ma Y, Luo X (2021) Landmarkminer: street-level network landmarks mining method for ip geolocation. ACM Trans Internet of Things 2(3):1–22

Li R, Yuan X, Yin M, Luo X (2024) Mobile ip geolocation based on district anchor without cooperation of users or internet service providers. IEEE/ACM Trans Netw 32(06):5507–5523

Lin J, Li C, Gong W, Song G, Fan L, Wang Z, Yang J (2024) Probegeo: a comprehensive landmark mining framework based on web content. IEEE/ACM Trans Netw 32(5):4398–4413

Liu C, Li R, Yuan F, Ding S, Liu Y, Luo X (2024) 6subpattern: target generation based on subpattern analysis for internet-wide ipv6 scanning. IEEE Trans Netw Serv Manage 21(4):3692–3710

Liu X, Wang X, Li C, Walker J, Tai W, Zhong T, Wang Y, Zhou F, Chen K (2025) Mapping the unseen: Robust ip geolocation through the lens of uncertainty quantification. Comput Netw 269:111405

Liu X, Tai W, Zhong T, Wang Y, Chen K, Zhou F (2026) Tracing paths, pruning noise: toward robust ip geolocation via topology-guided shaping and refinement. Inf Process Manag 63(3):104504

Liu C, Luo X, Yuan F, Liu F (2020) Rnbg: a ranking nodes based ip geolocation method. In: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE pp 80–84

Liu H, Zhang Y, Zhou Y, Zhang D, Fu X, Ramakrishnan K (2014) Mining checkins from location-sharing services for client-independent ip geolocation. In: IEEE INFOCOM 2014-IEEE Conference on Computer Communications, IEEE pp 619–627

Louizos C, Welling M, Kingma DP (2018) Learning sparse neural networks through l_0 regularization. In: International Conference on Learning Representations

Luo D, Cheng W, Yu W, Zong B, Ni J, Chen H, Zhang X (2021) Learning to drop: Robust graph neural network via topological denoising. In: Proceedings of the 14th ACM International Conference on Web Search and Data Mining. pp 779–787

Maddison CJ, Mnih A, Teh YW (2016) The concrete distribution: A continuous relaxation of discrete random variables.arXiv:1611.00712 arXiv preprint

Ortega-Velázquez MA, Martínez-Sala AS, Manzanares-López P, Cano M-D, Jara AJ (2025) Trust the source: A latency-based machine learning approach to accurate ip geolocation in internet. Comput Netw 272:111721

Padamanabban VN, Subramanian L (2001) Determining the geographic location of internet hosts. Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems. pp 324–325

Rong Y, Huang W, Xu T, Huang J (2020) Dropedge: towards deep graph convolutional networks on node classification. In: International Conference on Learning Representations. https://openreview.net/forum?id=Hkx1qkrKPr

Tai W, Chen B, Zhou, F, Zhong T, Trajcevski G, Wang Y, Chen K (2023) Trustgeo: Uncertainty-aware dynamic graph learning for trustworthy ip geolocation. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp 4862–4871

Wang Y, Burgener D, Flores M, Kuzmanovic A, Huang C (2011) Towards street-level client-independent ip geolocation. Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation. pp 365–379

Wang Z, Li H, Li Q, Li W, Zhu H, Sun L (2019) Towards ip geolocation with intermediate routers based on topology discovery. Cybersecurity 2(1):13

Wang X, Zhao D, Liu X, Zhang Z, Zhao T (2025) Neighborgeo: Ip geolocation based on neighbors. Comput Netw 257:110896

Wang Z, Zhou F, Zeng W, Trajcevski G, Xiao C, Wang Y, Chen K (2022) Connecting the hosts: Street-level ip geolocation with graph neural networks. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp 4121–4131

Wu G, Wang X (2025) A privacy-enhanced framework with deep learning for botnet detection. Cybersecurity 8(1):9

Xu H, Geng X, Liu J, Lu Z, Jiang B, Liu Y (2024) A novel approach for detecting malicious hosts based on re-gcn in intranet. Cybersecurity 7(1):69

Yang M, Liu C (2026) Xga-e: an explainability-enhanced graph neural network for network traffic anomaly detection. Cybersecurity 9(1):99

Yang K, Tai W, Li Z, Zhong T, Yin G, Wang Y, Zhou F (2024) Exploring self-explainable street-level ip geolocation with graph information bottleneck. In: ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, pp 7270–7274

Ye Y, Ji S (2021) Sparse graph attention networks. IEEE Trans Knowl Data Eng 35(1):905–916

Zhang F, Liu F, Luo X (2020) Geolocation of covert communication entity on the internet for post-steganalysis. EURASIP J Image Video Process 2020:1–10

Zhang S, Qu G, Zhang Z, Huang M, Jin H, Yang L (2025) Efficient and secure multi-party computation protocol supporting deep learning. Cybersecurity 8(1):46

Zhao T, Liu X, Zhang Z, Zhao D, Li N, Zhang Z, Wang X (2025) Hmcgeo: Ip region prediction based on hierarchical multi-label classification. IEEE Trans Netw Serv Manage 23:1913–1926

Zhao Z, Li R, Ding S, Luo X (2026) Sparsity is not an obstacle: An accurate and efficient ip geolocation framework based on graph transformer. Comput Netw 112287

Zhao T, Liu X, Zhang Z, Zhao D, Li N, Zhang Z, Wang X, Ji T (2025) Uncertaintygeo: A dirichlet network architecture for evaluating ip geolocation uncertainty. IEEE Transactions on Services Computing

Zheng C, Zong B, Cheng W, Song D, Ni J, Yu W, Chen H, Wang W (2020) Robust graph representation learning via neural sparsification. International Conference on Machine Learning. pp 11458–11468 (**PMLR**)
Zheng S, Liu Z, Zhu Z, Zhang X, Li J, Zhao Y (2025) Bikt: Unleashing the potential of gnns via bi-directional knowledge transfer. IEEE Trans Pattern Anal Mach Intell 1–15
Zilberman A, Offer A, Pincu B, Glickshtein Y, Kant R, Brodt O, Otung A, Puzis R, Shabtai A, Elovici Y (2024) A survey on geolocation on the internet. IEEE Commun Surveys Tutorials 27(5):3339–3381
Zu S, Luo X, Zhang F (2022) Ip-geolocater: a more reliable ip geolocation algorithm based on router error training. Front Comp Sci 16(1):161504
Zu S, Luo X, Wang L, Zhang F (2024) Gui: a geolocation method for unreachable ip. Comput J 67(5):1963–1978

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.